

그룹별 통계량과 상관관계

평균 옆에는 항상 흠어짐을, 흠어짐 옆에는 관계를

size / count var / std / median agg corr

2026년 8월 19일 (수) · 21일차 · 51차시

교안 14_01 마무리 · 14_02 그룹별 통계량 · 14_03 상관관계

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
14_01 빈도와 그룹 집계	실습 4~7 · groupby 복습 · size / count	마무리
14_02 그룹별 통계량	전 범위 · 실습 1~5	완주
14_03 상관관계와 통합 리포트	PART 01 상관관계 · 실습 1~2	절반
14_03 PART 02 집계 리포트	발견·해석·행동 · 실습 3~4	다음 시간

이 책을 읽는 법

배지 · 코드블록 · 이전 회차와의 연결

1. 분류 배지

모든 항목 이름 앞에는 그것이 **무엇인지**를 알려주는 색 배지가 붙습니다. 이름만 외우면 나중에 반드시 헛갈립니다.

배지	무엇	판별 신호	오늘의 예
내장함수	파이썬에 내장된 함수	앞에 점(.)이 없습니다	<code>len()</code> <code>range()</code>
메서드	객체에 붙는 함수	점 뒤에 있고 괄호가 붙습니다	<code>.std()</code> <code>.agg()</code>
속성	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>.columns</code> <code>.shape</code>
함수	라이브러리가 제공하는 함수	앞에 모듈 이름이 옵니다	<code>pd.cut()</code> <code>pd.read_csv()</code>
개념	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	분산 · 표준편차 · 상관계수
패턴	자주 쓰는 코드 흐름	여러 요소의 조합입니다	<code>groupby().agg().sort_values()</code>

개념

3초 판별법. ① 앞에 점(.)이 있습니까? 없으면 함수나 키워드입니다. ② 점이 있다면 뒤에 괄호가 붙습니까? 붙으면 메서드, 안 붙으면 속성입니다. 오늘 배울 `df.groupby("냉각기상태")["온도"].std()`를 이 기준으로 읽으면 `groupby`도 메서드, `std`도 메서드입니다. 반면 `df.columns`는 괄호가 없으니 속성입니다.

2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 상자 아래 가로줄 밑에 초록색 ▶로 시작하는 줄은 **그 코드를 실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 출력은 상상해서 쓴 것이 아니라 실제 실행 결과를 옮긴 것입니다.

예시

```
import pandas as pd

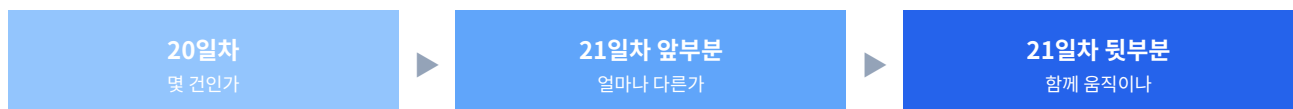
df = pd.read_csv("Data/14_hydraulic.csv", encoding="utf-8")
print(df.groupby("냉각기상태")["온도"].std().round(3))
```

```
▶ 냉각기상태
▶ 고장  3.760
▶ 저하  1.465
▶ 정상  0.356
▶ Name: 온도, dtype: float64
```

본문에서 이렇게 회색으로 칠해진 글자는 코드 조각입니다. **굵은 글씨**는 꼭 붙잡아야 할 문장입니다.

3. 이전 회차와의 연결

오늘은 **어제(20일차)** 배운 내용의 직접적인 연장선입니다. 어제 `value_counts()`로 "몇 건인가"를 세고 `groupby()`로 "그룹마다 평균이 얼마인가"를 구했다면, 오늘은 여기에 두 가지를 더합니다.



평균 하나만으로는 설비 상태를 판단할 수 없다는 사실을 배우고(분산·표준편차), 두 측정값이 서로 연동되는지 확인하는 도구를 배웁니다(상관계수). **어제의** `groupby`가 오늘 내내 뼈대로 쓰입니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	집계의 세 질문 — 몇 건인가 · 얼마나 다른가 · 함께 움직이나
STEP 1	groupby 복습과 size · count	어제 배운 3형식 되짚기, 행을 세는 세 가지 도구의 차이
STEP 2	실습 4·5·6 — 그룹 집계와 다중 키	기준을 바꾸고 함수를 바꾸고 기준을 겹치기
STEP 3	실습 7 — 같은 53에 이르는 세 갈래 길	len · size · value_counts 중 무엇을 쓸 것인가
STEP 4	평균만 보면 놓치는 것	대푯값의 한계, 분산과 표준편차
STEP 5	양 끝값과 중앙값	min · max · count · median으로 치우침 진단
STEP 6	agg — 여러 통계를 한 번에	리스트 방식과 이름 붙이기 방식
STEP 7	진단표 만들기과 그 함정	실습 1~5, 그리고 "편차가 안 줄었다"는 발견
STEP 8	상관관계 — 함께 움직이는가	corr, 부호와 절댓값, 상관 행렬, 상관 ≠ 인과
STEP 9	실습 1·2 — 강한 상관 쌍 찾기	상관 행렬 읽기, 이중 반복문으로 자동 추출
부록 A	치트시트	오늘 나온 모든 도구를 한 표에
부록 B	오류·함정 사전	제출 코드 교정 · 교안 예상 결과 대조
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · 14_03 PART 02 예고

개념

오늘의 한 문장. "평균은 가운데가 어디인지만 알려줄 뿐, 얼마나 퍼져 있는지는 알려주지 않는다." 이 문장이 오늘 수업의 4장부터 7장까지를 관통합니다. 그리고 8장부터는 질문이 한 번 더 바뀝니다. 하나의 열을 요약하는 데서 벗어나, 두 열 사이의 관계를 봅니다.

오늘의 지도

집계가 답하는 세 가지 질문

0-1. 데이터 앞에서 던지는 세 가지 질문

설비 데이터 한 장을 받았다고 해 봅시다. 120줄짜리 유압 시스템 측정 기록입니다. 이 표를 그대로 눈으로 읽는 사람은 없습니다. 우리는 **질문을 던지고 그 질문에 맞는 도구를 고릅니다**. 지난 시간과 오늘을 합치면 그 질문은 세 단계로 깊어집니다.

단계	질문	도구	언제 배웠나
1	몇 건인가	<code>value_counts()</code> <code>pd.cut()</code>	20일차
2	그룹마다 평균이 얼마인가	<code>groupby().mean()</code>	20일차 끝
3	그 평균을 믿어도 되는가	<code>var()</code> <code>std()</code> <code>agg()</code>	오늘 4~7장
4	두 값이 함께 움직이는가	<code>corr()</code>	오늘 8~9장

3단계가 오늘 수업의 핵심입니다. 어제 우리는 냉각기상태별 평균 온도가 고장 54.67도, 저하 45.46도, 정상 35.89도라는 결과를 얻었습니다. 그런데 **이 세 숫자만으로 "고장 라인이 뜨겁다"고 결론 내려도 되는지**는 아직 확인하지 않았습니다. 오늘 그 확인 방법을 배웁니다.

0-2. 오늘 다루는 데이터

세 교안 모두 같은 유압 시스템 데이터를 씁니다. 다만 상관관계 실습에서는 품질검사 데이터가 추가로 등장합니다.

파일	행 × 열	범주형 열	연속형 열
14_hydraulic.csv	120 × 8	냉각기상태 · 운전부하 · 밸브상태 · result	온도 · 진동 · 압력 · 냉각효율
14_hydraulic_qc.csv	200 × 11	검사결과	지표01 ~ 지표10

설비 적용

유압 시스템은 **60초 부하 사이클**을 반복하며 온도 · 진동 · 압력을 측정합니다. 각 사이클에는 냉각기 상태, 밸브 상태, 운전 부하가 함께 기록됩니다. 그래서 이 데이터는 **범주형 열로 그룹을 나누고 연속형 열의 통계를 내는 구조에** 딱 맞습니다. 이것이 `groupby`가 이 데이터에서 자연스럽게 쓰이는 이유입니다.

한 가지 미리 짚어 둘 점이 있습니다. 두 파일 모두 **결측값이 하나도 없습니다**. 어제 다이캐스팅 데이터에서 결측 14건 때문에 드모르간 법칙이 깨져 보였던 것과 달리, 오늘은 그런 함정이 없습니다. 대신 `size()`와 `count()`의 차이를 설명할 때 결측이 없다는 사실이 오히려 **차이를 눈으로 확인할 수 없게** 만듭니다. 그래서 1장에서 결측을 일부러 만들어 확인해 보겠습니다.

0-3. 오늘 등장하는 통계 용어 미리보기

용어	한 줄 정의	단위	왜 필요한가
평균 mean	모든 값을 더해 개수로 나눈 값	원래 단위	데이터 전체를 한 숫자로 요약
분산 var	평균에서 떨어진 거리를 제곱해 평균 낸 값	단위의 제곱	흩어진 정도를 수치화
표준편차 std	분산에 제곱근을 씌운 값	원래 단위	흩어짐을 원래 단위로 해석
중앙값 median	크기순으로 줄 세웠을 때 한가운데 값	원래 단위	극단값에 흔들리지 않는 대푯값
상관계수 corr	두 값이 함께 움직이는 정도	없음 (-1 ~ 1)	두 측정값의 연동 확인

비유

평균은 반 평균 점수, 표준편차는 성적 편차입니다. 두 반의 평균이 똑같이 78점이어도 한 반은 전원이 75~81점이고 다른 반은 40~100점에 흩어져 있을 수 있습니다. 담임 입장에서 두 반은 전혀 다른 반입니다. 설비도 마찬가지로입니다. **평균 온도가 같아도 편차가 크면 그 설비는 위험합니다.**

STEP 1

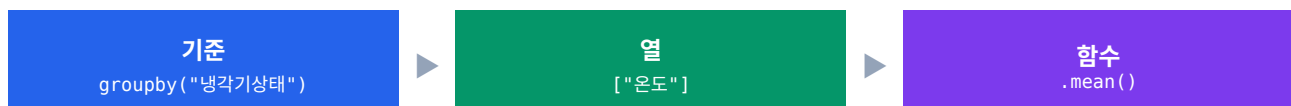
groupby 복습과 size · count

행을 세는 세 가지 도구

강사님은 오늘 수업을 어제 배운 groupby를 다시 만들어 보는 것으로 시작하셨습니다. "어제까지 했던 내용 다시 살펴보겠습니다"라며 코드를 처음부터 다시 치셨습니다. 여기서는 그 복습 코드를 그대로 따라가면서, 어제 미처 짚지 못한 size()와 count()를 새로 배웁니다.

1-1. groupby 3형식 되짚기

groupby는 항상 **기준** → **열** → **함수** 세 조각으로 이루어집니다. 이 순서만 기억하면 어떤 집계든 쓸 수 있습니다.



메서드 `.groupby()` 기준 열의 값이 같은 행끼리 묶기

정의 지정한 열의 값이 같은 행들을 하나의 그룹으로 묶어 **GroupBy 객체**를 만듭니다. 이 객체 자체는 아직 아무 계산도 하지 않은 상태입니다.

형태 `df.groupby("기준열")["대상열"].집계함수()`

```
import pandas as pd

df = pd.read_csv("Data/14_hydraulic.csv", encoding="utf-8")

# 냉각기상태마다 평균 온도 - 소수점 이하 2자리
print(df.groupby("냉각기상태")["온도"].mean().round(2))
```

```
▶ 냉각기상태
▶ 고장  54.67
▶ 저하  45.46
▶ 정상  35.89
▶ Name: 온도, dtype: float64
```

왜 쓰나 설비마다, 라인마다, 교대마다 값이 어떻게 다른지 비교하려면 먼저 같은 것끼리 묶어야 합니다. groupby 없이 이 결과를 내려면 조건 필터링을 상태 수만큼 반복해야 합니다.

응용 집계 결과도 Series이므로 뒤에 `.sort_values()`를 이어 붙여 순위를 낼 수 있습니다. 점을 계속 이어 붙이는 이 방식을 **메서드 체이닝**이라고 부릅니다.

```
print(df.groupby("운전부하")["진동"].mean().round(3))

# 정렬까지 이어 붙이기
print(df.groupby("밸브상태")["진동"].mean().round(3).sort_values(ascending=False))
```

```
▶ 운전부하
▶ 고부하  0.602
▶ 저부하  0.629
▶ Name: 진동, dtype: float64
▶
▶ 밸브상태
▶ 심각  0.629
▶ 자연  0.621
▶ 경미  0.617
▶ 정상  0.609
```

개념

groupby의 결과 인덱스는 **기준 열의 값**이 됩니다. 위 출력에서 왼쪽 줄에 있는 고장·저하·정상은 데이터의 행 번호가 아니라 냉각기상태의 값들입니다. 그래서 출력 맨 위에 냉각기상태라는 인덱스 이름이 따로 찍힙니다.

1-2. 그룹마다 몇 개인가 — size

강사님이 던진 질문은 이것이었습니다. "**냉각기상태별로 얼마나 많은 항목이 있을까요?**" 어제 배운 방법으로도 답할 수 있습니다. 다만 상태 수만큼 코드를 반복해야 합니다.

```
비효율적인 방법

# 각 상태별로 개수를 따로따로 계산하는 방법
print(len(df[df["냉각기상태"] == "고장"]))
print(len(df[df["냉각기상태"] == "저하"]))
print(len(df[df["냉각기상태"] == "정상"]))
```

```
▶ 40
▶ 40
▶ 40
```

상태가 세 개라 세 줄이면 되지만, **상태가 스무 개라면 스무 줄**을 써야 합니다. 게다가 상태 값이 무엇인지 미리 알아야 합니다. 이 반복을 없애는 것이 `size()`입니다.

메서드 `.size()` 그룹마다 행이 몇 개인지 세기

정의 GroupBy 객체에 붙여 **각 그룹의 행 개수**를 Series로 돌려줍니다. 대상 열을 지정할 필요가 없습니다.

형태 `df.groupby("기준열").size()`

```
# 각 상태별로 갯수를 따로따로 계산하는 것은 비효율적 - size 활용
```

```
print(df.groupby("냉각기상태").size())
```

```
▶ 냉각기상태
▶ 고장    40
▶ 저하    40
▶ 정상    40
▶ dtype: int64
```

왜 쓰나 세 줄이 한 줄이 됩니다. 더 중요한 것은 **상태 값을 미리 몰라도 된다는** 점입니다. 데이터에 어떤 값이 있든 전부 알아서 세어 줍니다.

응용 groupby에 리스트를 넘기면 **조합별 건수**를 한 번에 셀 수 있습니다. 아래는 냉각기상태와 운전부하를 겹친 결과입니다. 곧 실습 6에서 다시 보겠지만, 여기서 이미 이상한 점이 하나 보입니다.

```
print(df.groupby(["냉각기상태", "운전부하"]).size())
```

```
▶ 냉각기상태 운전부하
▶ 고장   고부하   17
▶   저부하   23
▶ 저하   고부하   3
▶   저부하   37
▶ 정상   고부하   40
▶ dtype: int64
```

개념

`size()`에는 **대상 열을 붙이지 않습니다**. 강사님 표현대로 "각 그룹 안에 몇 개의 항목이 있는지만 세라"는 뜻이므로 세부 컬럼 이름을 언급할 필요가 없습니다. `df.groupby("냉각기상태")["온도"].size()`처럼 열을 붙여도 결과는 같지만, 굳이 붙일 이유가 없습니다.

주의

위 응용 결과에서 **정상-저부하 조합이 아예 없습니다**. $17+23+3+37+40 = 120$ 으로 전체 행 수와 맞으니 빠진 것이 아니라 원래 없는 조합입니다. `size()`는 **존재하지 않는 조합은 출력하지 않습니다**.

1-3. 결측을 빼고 세는가 — count

count()도 개수를 셉니다. 그런데 세는 대상이 다릅니다. 강사님이 강조하신 지점이 정확히 여기입니다.

메서드 .count() 결측이 아닌 값이 몇 개인지 세기

정의 각 그룹에서 **결측값(NaN)이 아닌 실제 값의 개수**를 셉니다. 대상 열을 반드시 지정해야 의미가 있습니다.

형태 df.groupby("기준열")["대상열"].count()

```
# 결측이 없는 원본에서는 size와 결과가 같다
print(df.groupby("냉각기상태")["온도"].count())
```

```
▶ 냉각기상태
▶ 고장    40
▶ 저하    40
▶ 정상    40
▶ Name: 온도, dtype: int64
```

왜 쓰나 평균이나 표준편차는 **실제 값의 개수**로 나눕니다. 그러니 결측이 섞여 있으면 "이 평균이 몇 개를 근거로 나온 것인지" 확인해야 합니다. count()가 그 확인 도구입니다.

응용 오늘 데이터는 결측이 하나도 없어서 size()와 count()가 똑같이 나옵니다. 그래서 차이를 눈으로 보려면

결측을 일부러 만들어야 합니다. 아래는 고장 그룹의 온도 3개를 결측으로 바꾼 뒤 비교한 결과입니다.

```
import numpy as np

test = df.copy()
test.loc[test.index[:3], "온도"] = np.nan    # 앞 3행의 온도를 결측으로

print(test.groupby("냉각기상태").size())      # 행 수 그대로
print(test.groupby("냉각기상태")["온도"].count()) # 결측 제외
```

```
▶ 냉각기상태
▶ 고장    40    ← size는 40 그대로
▶ 저하    40
▶ 정상    40
▶
▶ 냉각기상태
▶ 고장    37    ← count는 3 줄어듦
▶ 저하    40
▶ 정상    40
```

주의

강사님 말씀 그대로 옮기면 "결측 개수도 포함되어 있다"입니다. `size()`는 결측이 있든 없든 행이 있으면 셉니다. 그래서 "이 그룹에 데이터가 몇 줄 있나"를 물을 때는 `size()`, "그중 실제로 계산에 쓸 수 있는 값이 몇 개인가"를 물을 때는 `count()`를 씁니다.

1-4. size · count · value_counts 3자 비교

행을 세는 도구가 셋이 되었습니다. 셋 다 숫자를 돌려주지만 세는 대상과 결과 모양이 다릅니다. 이 표가 오늘 1장의 결론입니다.

구분			
무엇을 세나	그룹의 행 수	결측 아닌 값의 수	값별 등장 횟수
결측 처리	포함해서 셉	빼고 셉	빼고 셉
대상 열	불필요	필요	Series에 직접
정렬	인덱스 순	인덱스 순	빈도 내림차순
붙는 곳	GroupBy	GroupBy 또는 열	Series (열 하나)
없는 조합	출력 안 함	출력 안 함	출력 안 함

자주 하는 실수

`value_counts()`를 수치형 열에 그대로 쓰는 것입니다. 오늘 데이터의 온도 열은 고유값이 62개, 진동은 80개, 압력은 101개입니다. `df["압력"].value_counts()`를 실행하면 101줄짜리 무의미한 목록이 쏟아집니다. 수치형은 어제 배운 `pd.cut()`으로 구간을 묶은 뒤에 세야 합니다. 이 실수는 실제로 제출 코드에 있었고, 부록 B에 교정을 실었습니다.

STEP 2

실습 4 · 5 · 6

기준을 바꾸고, 함수를 바꾸고, 기준을 겹치기

교안 14_01의 실습 4부터 7까지는 어제 시간이 모자라 넘긴 부분입니다. 오늘 앞 시간에 이 넷을 모두 풀었습니다. 강사님은 화면을 반으로 나눠 문제와 코드를 나란히 놓고 진행하셨습니다.

2-1. 실습 4 — groupby로 그룹 집계

항목	내용
목표	기준 → 열 → 함수 순으로 그룹별 통계 구하기
단계 1	라인으로 그룹을 나눠 압력 열의 평균 집계
단계 2	집계 함수를 바꿔 설비별 최고 온도 확인
단계 3	size로 교대별 측정 건수까지 확인
예상 결과	라인별 평균 압력, 설비별 최고 온도, 교대별 건수 출력

개념

교안은 일반적인 공장 용어로 문제를 냈지만 실제 데이터의 열 이름은 다릅니다. **라인 = 냉각기상태 · 설비 = 밸브상태 · 교대 = 운전부하**로 바꿔 읽어야 합니다. 이 대응을 먼저 정리하지 않으면 문제를 풀 수 없습니다.

Practice/14_01_example.py — 실습 4

```
import pandas as pd
import os

path_4 = os.path.join("Data", "14_hydraulic.csv")
df_4 = pd.read_csv(path_4, encoding="utf-8")

# 1. 라인으로 그룹을 나눠 압력 열의 평균 집계 (라인 = 냉각기상태)
print(df_4.groupby("냉각기상태")["압력"].mean().round(2))

# 2. 집계 함수를 바꿔 설비별 최고 온도 확인 (설비 = 밸브상태)
print(df_4.groupby("밸브상태")["온도"].max())

# 3. size로 교대별 측정 건수까지 확인 (교대 = 운전부하)
```

```
print(df_4.groupby("운전부하").size())
```

```
▶ 냉각기상태
▶ 고장 163.68
▶ 저하 158.49
▶ 정상 160.84
▶ Name: 압력, dtype: float64
▶
▶ 밸브상태
▶ 경미 57.1
▶ 심각 57.6
▶ 정상 57.8
▶ 자연 57.5
▶ Name: 온도, dtype: float64
▶
▶ 운전부하
▶ 고부하 60
▶ 저부하 60
▶ dtype: int64
```

세 출력을 차례로 읽어 봅시다. **압력은 라인별 차이가 거의 없습니다.** 158.49에서 163.68 사이, 5 정도 차이입니다. 반면 어제 본 온도는 35.89에서 54.67까지 19도나 벌어졌습니다. 같은 기준으로 나눠도 **어떤 열은 같리고 어떤 열은 안 같립니다.**

두 번째 출력은 더 흥미롭습니다. **밸브상태로 나눈 최고 온도가 네 그룹 모두 57.1~57.8도로 사실상 같습니다.** 밸브가 정상이든 심각이든 최고 온도는 똑같이 57도대라는 뜻입니다. 이 관찰은 7장에서 결정적인 발견으로 이어집니다.

주의

`max()`는 `round()`를 붙이지 않았습니니다. 이미 원본 값이 소수점 한 자리라 반올림할 것이 없기 때문입니다.

`mean()`은 나눗셈이라 소수점이 길게 나오지만, `max()`는 원본에 있던 값을 그대로 꺼내오는 것이라 **자릿수가 늘지 않습니다.**

2-2. 실습 5 — 그룹별 평균 비교와 정렬

실습 5는 집계 결과에 정렬을 이어 붙이는 연습입니다. 강사님은 **"앞선 평균 결과에 맞춰서 처음 언급돼야 될 게 뭐겠어요? 심각이죠"**라며 정렬 후 순서를 미리 예측하게 하셨습니다.

Practice/14_01_example.py — 실습 5

```
# 1. 설비로 그룹을 나눠 진동 평균 집계
```

```
print(df_5.groupby("밸브상태")["진동"].mean().round(3))
```

```
# 2. 집계 결과에 정렬을 이어 붙여 내림차순으로 정렬
```

```
print(df_5.groupby("밸브상태")["진동"].mean().round(3).sort_values(ascending=False))
```

```
▶ 밸브상태
```

```
▶ 경미 0.617
```

```
▶ 심각 0.629
```

```
▶ 정상 0.609
```

```
▶ 자연 0.621
```

```
▶ Name: 진동, dtype: float64
```

```
▶
```

```
▶ 밸브상태
```

```
▶ 심각 0.629 ← 가장 큼
```

```
▶ 자연 0.621
```

```
▶ 경미 0.617
```

```
▶ 정상 0.609
```

```
▶ Name: 진동, dtype: float64
```

정렬 전 출력은 **경미 → 심각 → 정상 → 자연** 순인데, 이것은 크기 순이 아니라 **한글 가나다 순**입니다. `groupby`가 기준 값을 자동으로 정렬해 인덱스로 삼기 때문입니다. 여기에 `.sort_values(ascending=False)`를 붙이면 비로소 **값의 크기 순**이 됩니다.

주의

네 그룹의 진동 평균이 0.609에서 0.629까지, **차이가 0.02밖에 안 됩니다**. 정렬해서 순위를 매기면 심각이 1위처럼 보이지만, 이 정도 차이를 근거로 "심각 밸브의 진동이 크다"고 말하기는 어렵습니다. **정렬은 순위를 보여줄 뿐 차이가 의미 있는지는 알려주지 않습니다**. 이것을 판단하려면 4장에서 배울 표준편차가 필요합니다.

2-3. 실습 6 — 여러 기준 조합 그룹

기준 열을 리스트로 넘기면 그룹이 **중첩**됩니다. 리스트에 적은 순서대로 바깥 그룹과 안쪽 그룹이 정해집니다.

Practice/14_01_example.py — 실습 6

```
# 1. 라인과 교대 두 기준을 묶어 진동 평균 집계
```

```
print(df_6.groupby(["냉각기상태", "운전부하"])["진동"].mean().round(3))
```

```
# 2. 같은 두 기준으로 size를 구해 조합별 측정 건수 확인
```

```
print(df_6.groupby(["냉각기상태", "운전부하"]).size())
```

```
▶ 냉각기상태 운전부하
▶ 고장 고부하 0.726
▶ 저부하 0.660
▶ 저하 고부하 0.616
▶ 저부하 0.610
▶ 정상 고부하 0.549
▶ Name: 진동, dtype: float64
▶
▶ 냉각기상태 운전부하
▶ 고장 고부하 17
▶ 저부하 23
▶ 저하 고부하 3 ← 단 3건
▶ 저부하 37
▶ 정상 고부하 40
▶ dtype: int64
```

출력에서 왼쪽 열이 비어 보이는 줄이 있습니다. 이는 값이 없다는 뜻이 아니라 **바로 위와 같은 값이라 생략된** 것입니다.

이런 이중 인덱스를 **MultiIndex**라고 부릅니다.

자주 하는 실수

건수가 적은 조합의 평균을 그대로 믿는 것입니다. 저하-고부하 조합은 단 3건입니다. 3건으로 낸 평균 0.616은 한 건만 바뀌어도 크게 흔들립니다. 강사님도 이 지점에서 **"해당하는 개수로 평균을 계산할 때 오차라는 게 생길 수 있다, 해석의 문제가 생길 수 있다"**고 강조하셨습니다. **평균을 볼 때는 반드시 그 옆에 건수를 함께 놓아야 합니다.**

그리고 목록에 **정상-저부하 조합이 아예 없습니다.** 다섯 줄의 합은 $17+23+3+37+40 = 120$ 으로 전체 행 수와 정확히 맞으니 누락이 아니라 원래 없는 조합입니다. 즉 이 데이터에서 **냉각기가 정상일 때는 항상 고부하로만 운전되었습니다.** `groupby`는 존재하지 않는 조합을 만들어 내지 않습니다.

STEP 3

실습 7 — 같은 53에 이르는 세 갈래 길

도구가 아니라 문제에 맞는 답을 고르기

실습 7은 빈도와 그룹 집계를 종합하는 문제입니다. 여기서 강사님이 가장 오래 붙잡으신 대목은 **같은 숫자를 얻는 세 가지 방법의 차이**였습니다.

3-1. 밸브 구성 파악 — 건수와 비율

Practice/14_01_example.py — 실습 7

```
# value_counts로 설비 구성과 정상·고장 비율 파악
print(df_7["밸브상태"].value_counts())
print(df_7["밸브상태"].value_counts(normalize=True).round(3))
```

```
▶ 밸브상태
▶ 정상  61
▶ 지연  20
▶ 경미  20
▶ 심각  19
▶ Name: count, dtype: int64

▶ 밸브상태
▶ 정상  0.508
▶ 지연  0.167
▶ 경미  0.167
▶ 심각  0.158
▶ Name: proportion, dtype: float64
```

밸브가 정상인 사이클이 61건으로 전체의 **50.8%**입니다. 나머지 절반이 경미·지연·심각으로 거의 균등하게 나뉘어 있습니다. 어제 배운 `normalize=True`가 여기서 다시 쓰입니다. **건수만 보면 "정상이 제일 많다"**에서 끝나지만, **비율로 보면 "절반이 정상이 아니다"**라는 문장이 나옵니다.

3-2. 같은 53, 다른 세 가지 코드

문제는 "고장 행만 걸러 고장 건수 집계"입니다. 강사님은 이 한 문제에 세 가지 답을 나란히 써 보이셨습니다. **"정답이 하나는 아니에요"**라고 하시면서요.

세 가지 답

아래 세 가지는 같은 53을 주지만 의미가 다름

```
print(len(df_7[df_7["result"] == "고장"])) # 방법 1
```

```
print(df_7.groupby("result").size()) # 방법 2
```

```
print(df_7["result"].value_counts()) # 방법 3
```

```
▶ 53
▶
▶ result
▶ 고장 53
▶ 정상 67
▶ dtype: int64
▶
▶ result
▶ 정상 67
▶ 고장 53
▶ Name: count, dtype: int64
```

방법	코드	돌려주는 것	문제와의 부합
1	<code>len(df[조건])</code>	고장 행 수 하나 (정수)	가장 부합
2	<code>groupby().size()</code>	모든 값별 행 수 (Series)	덤이 붙음
3	<code>value_counts()</code>	모든 값별 빈도, 내림차순 (Series)	덤이 붙음

개념

강사님 결론은 명확했습니다. "문제는 고장 행만 걸러서라고 돼 있으니 문제와 가장 부합하는 건 이거겠쥬." 방법 2와 3은 정상 67건까지 함께 돌려줍니다. 필요 없는 정보가 딸려 오는 것이지요. 다만 "고장이 몇 건인지"가 아니라 "각 상태가 몇 건씩인지"가 궁금하다면 방법 2와 3이 더 낫습니다. 도구에 우열이 있는 것이 아니라 문제에 맞는 도구가 따로 있습니다.

방법 2와 3의 차이도 짚어 둘 만합니다. `size()`는 **인덱스 순**(고장 → 정상)으로, `value_counts()`는 **빈도 내림차순**(정상 67 → 고장 53)으로 정렬합니다. 순위를 보고 싶으면 `value_counts()`, 값 순으로 보고 싶으면 `size()`입니다.

3-3. 두 열을 한 번에 집계하기

패턴 `groupby(...)[["열1", "열2"]]` 여러 열을 동시에 집계

정의 대상 열 자리에 **리스트**를 넣으면 여러 열의 통계를 한 번에 구해 **데이터프레임**으로 돌려줍니다.

형태 `df.groupby("기준열")[["열1", "열2"]].집계함수()`

```
# 열을 리스트로 넘기면 두 열을 따로 집계하지 않고 한 번에 처리 가능
print(df_7.groupby("냉각기상태")[["온도", "진동"]].mean().round(2))
```

```
▶   온도  진동
▶ 냉각기상태
▶ 고장   54.67  0.69
▶ 저하   45.46  0.61
▶ 정상   35.89  0.55
```

왜 쓰나 열마다 따로 코드를 쓰면 출력이 따로 나와서 **눈으로 짝을 맞춰 읽어야** 합니다. 리스트로 넘기면 처음부터 표 하나로 나오니 비교가 훨씬 쉽습니다.

응용 대괄호 하나와 두 개의 차이는 어제 배운 것과 같습니다. **대괄호 하나는 Series, 두 개는 데이터프레임**입니다. 아래에서 결과 타입이 어떻게 달라지는지 확인해 보십시오.

```
a = df.groupby("냉각기상태")["온도"].mean()
b = df.groupby("냉각기상태")[["온도"]].mean()

print(type(a))
print(type(b))

▶ <class 'pandas.core.series.Series'>
▶ <class 'pandas.core.frame.DataFrame'>
```

개념

출력에서 온도는 54.67처럼 소수 둘째 자리, 진동은 0.69처럼 보입니다. round(2)를 표 전체에 걸었기 때문입니다. **진동은 원래 소수 셋째 자리까지 있는 값이라 자릿수를 잃었습니다.** 열마다 다른 자릿수가 필요하면 6장에서 배울 agg가 답입니다.

STEP 4

평균만 보면 놓치는 것

대푯값의 한계 · 분산 · 표준편차

여기서부터 교안 14_02입니다. 그리고 오늘 수업에서 **가장 중요한 개념적 전환**이 일어나는 지점이기도 합니다. 지금까지 우리는 그룹마다 평균을 구하고 그 평균을 비교했습니다. 그런데 그 비교가 정당한지는 한 번도 묻지 않았습니

4-1. 평균이라는 대푯값

개념 평균 (mean) 흩어진 값들을 하나로 압축한 기준점

정의 모든 값을 더해 개수로 나눈 값입니다. 수십·수백 개의 측정값을 **한 숫자로 요약해 비교의 기준점**으로 삼습니다.

형태 `df["열"].mean()` | `df.groupby("기준")["열"].mean()`

```
# 전체 평균 온도
print(df["온도"].mean().round(3))

# 냉각기상태별 평균 온도
print(df.groupby("냉각기상태")["온도"].mean().round(3))

▶ 45.339
▶
▶ 냉각기상태
▶ 고장 54.668
▶ 저하 45.465
▶ 정상 35.885
▶ Name: 온도, dtype: float64
```

왜 쓰나 120줄을 눈으로 훑어서는 "고장 라인이 더 뜨겁다"는 사실을 알 수 없습니다. 평균 하나로 압축하면 **19도 차이**가 즉시 보입니다.

응용 평균은 **모든 값을 골고루 반영**하기 때문에, 실제로는 존재하지 않는 값이 나올 수 있습니다. 교안의 표현이 정확합니다. 가족 평균 인원이 2.5명이라 해도 2.5명인 가족은 없습니다. 평균은 **데이터를 요약한 가상의 기준점**입니다.

```
# 평균값이 실제 데이터에 존재하는지 확인
m = df["온도"].mean()
print(round(m, 3))
print((df["온도"] == m).sum()) # 평균과 정확히 같은 행이 몇 개인가
```

▶ 45.339

▶ 0 ← 평균과 정확히 같은 측정값은 하나도 없다

개념

전체 평균 45.339는 저하 라인의 평균 45.465와 거의 같습니다. 그렇다고 **"저하 라인이 전체를 대표한다"**고 말하면 **안 됩니다**. 고장(54.67)이 위로, 정상(35.89)이 아래로 똑같이 끌어당겨서 우연히 가운데에 온 것뿐입니다.

4-2. 평균이 같은 두 설비

교안이 던지는 질문은 이것입니다. **"두 라인의 평균이 똑같이 78도라면, 두 라인은 같은 상태일까요?"** 강사님도 이 예시를 길게 설명하셨습니다. 78도면 꽤 뜨거운 편이지만, 설비 종류에 따라 정상일 수도 있습니다.

구분	설비 가	설비 나
평균 온도	78도	78도
측정 범위	76 ~ 80도	50 ~ 105도
흠어짐	작음 — 안정	큼 — 불안정
현장 판단	정상 가동	즉시 점검 대상

설비 적용

평균이 정상이어도 값이 흔들리면 위험 신호입니다. 설비 나 는 어떤 사이클에서는 50도, 다른 사이클에서는 105도를 찍습니다. 평균으로 뭉개면 78도지만, 실제로는 냉각이 제대로 안 되고 있거나 부하가 들쭉날쭉하다는 뜻입니다. **평균 옆에는 항상 "얼마나 퍼져 있는가"를 함께 놓아야 합니다.**

4-3. 분산 — 흠어짐을 수치로

메서드 `.var()` 평균에서 떨어진 거리의 제곱 평균

정의 각 값이 평균에서 얼마나 떨어져 있는지를 **제공해서 평균 낸 값**입니다. `variance`의 줄임말입니다.

형태 `df.groupby("기준열")["대상열"].var()`

```
# 분산 구하기
print(df.groupby("냉각기상태")["온도"].var().round(3))
```

```
▶ 냉각기상태
▶ 고장 14.135 ← 가장 들쭉날쭉
▶ 저하 2.147
▶ 정상 0.126 ← 가장 안정
▶ Name: 온도, dtype: float64
```

왜 쓰나 "고장 라인이 더 뜨겁다"에서 한 걸음 나아가 "고장 라인은 더 뜨거우면서 동시에 더 불안정하다"는 문장을 쓸 수 있게 됩니다. 정비 우선순위를 정할 때 이 두 번째 문장이 훨씬 강력합니다.

응용 평균과 분산을 함께 읽으면 세 라인의 성격이 완전히 갈립니다. 강사님이 코드 아래에 직접 정리해 두신 세 줄이 이 표의 원형입니다.

```
import pandas as pd

m = df.groupby("냉각기상태")["온도"].mean()
v = df.groupby("냉각기상태")["온도"].var()
print(pd.DataFrame({"평균": m.round(2), "분산": v.round(3)}))
```

```
▶   평균   분산
▶ 냉각기상태
▶ 고장 54.67 14.135
▶ 저하 45.46  2.147
▶ 정상 35.89  0.126
```

개념

왜 제곱을 할까요. 평균에서 떨어진 거리를 그냥 더하면 **평균보다 큰 값과 작은 값이 서로 상쇄되어 항상 0이 됩니다.** 제곱하면 음수가 사라지므로 상쇄되지 않습니다. 이것이 제곱하는 이유입니다.

주의

분산의 치명적인 약점은 **단위**입니다. 거리를 제곱했으니 단위도 제곱됩니다. 온도의 분산 14.135는 "14.135 도"가 아니라 "**14.135 도의 제곱**"입니다. 도의 제곱이라는 단위는 현장에서 아무 의미도 없습니다.

4-4. 표준편차 — 원래 단위로 돌아오기

메서드 `.std()` 분산에 제곱근을 씌워 원래 단위로

정의 분산의 제곱근입니다. 제곱해서 부풀렸던 단위를 다시 원래대로 되돌립니다. `standard deviation`의 줄임말입니다.

형태 `df.groupby("기준열")["대상열"].std()`

```
# 표준편차 구하기
print(df.groupby("냉각기상태")["온도"].std().round(3))
```

```
▶ 냉각기상태
▶ 고장    3.760
▶ 저하    1.465
▶ 정상    0.356
▶ Name: 온도, dtype: float64
```

왜 쓰나 3.760은 도 단위입니다. 바로 해석됩니다. "고장 라인의 개별 온도는 평균 54.67도에서 평균적으로 약 3.8도쯤 떨어져 있다"는 문장이 곧바로 나옵니다. 분산의 14.135로는 이런 문장을 쓸 수 없습니다.

응용 표준편차를 제공하면 정확히 분산이 됩니다. 실습 1이 확인하라고 요구한 것이 바로 이 관계입니다. **부동소수점 오차 때문에 자릿수를 넉넉히 잡고 비교**해야 정확히 같음을 확인할 수 있습니다.

```
s = df.groupby("냉각기상태")["온도"].std()
v = df.groupby("냉각기상태")["온도"].var()
print(pd.DataFrame({"std": s.round(3), "std**2": (s ** 2).round(3), "var":
v.round(3)}))
```

```
▶          std  std**2      var
▶ 냉각기상태
▶ 고장    3.760   14.135   14.135
▶ 저하    1.465    2.147    2.147
▶ 정상    0.356    0.126    0.126
```

개념

평균과 표준편차를 한 쌍으로 묶으면 이렇게 읽습니다. **고장 54.67 ± 3.76 · 저하 45.46 ± 1.47 · 정상 35.89 ± 0.36**. 이 표기 하나로 "어디쯤에" "얼마나 모여 있는지"를 동시에 말할 수 있습니다.

4-5. 분산과 표준편차, 무엇을 외울 것인가

구분		
의미	흩어진 정도	흩어진 정도 (같음)
단위	단위의 제곱 — 해석 불가	원래 단위 — 바로 해석
관계	std^{**2}	$\text{var}^{**0.5}$
온도 예시	14.135 (도의 제곱)	3.760 (도)
실무 사용	드뭄	훨씬 많이 씀

개념

교안의 문장을 그대로 옮깁니다. "둘 중 하나만 외운다면 표준편차." 실무에서 표준편차를 압도적으로 많이 쓰는 이유는 단 하나, 단위가 원래 데이터와 같아서 바로 해석되기 때문입니다. 분산은 표준편차를 계산하는 중간 단계라고 이해하면 충분합니다.

4-6. 설비 상태 4분면

평균과 표준편차를 한 쌍으로 묶으면 설비를 네 가지로 진단할 수 있습니다. 이것이 오늘 배운 두 통계량을 현장에서 쓰는 방식입니다.

사분면	평균	표준편차	상태	대응
① 이상적 안정	정상	작음	적절한 온도에서 안정적	현행 유지
② 불안정 주의	정상	큼	평균은 괜찮은데 요동침	모니터링 강화
③ 일관된 과부하	높음	작음	꾸준히 높게 유지	설계 조건 재검토
④ 고위험	높음	큼	뜨거우면서 요동침	즉시 점검

오늘 데이터의 세 라인을 이 틀에 넣어 보겠습니다. 고장 라인(54.67 ± 3.76)은 평균도 가장 높고 편차도 가장 큼니다. 4사분면, 즉 고위험입니다. 반대로 정상 라인(35.89 ± 0.36)은 평균이 가장 낮으면서 편차도 0.36으로 거의 없습니다. 1사분면, 가장 이상적입니다.

주의

교안 p14는 세 라인을 A·B·C로 부르면서 "C 라인이 가장 불안정"이라고 적었지만, 실제 숫자를 대입하면 C에 해당하는 값(평균 35.89, 표준편차 0.36)이 가장 안정적입니다. 라벨이 어긋난 것으로 보입니다. 교안의 결론 문장보다 실제 계산 결과를 우선하십시오. 자세한 대조는 부록 B에 있습니다.

STEP 5

양 끝값과 중앙값

min · max · count · median

평균과 표준편차 다음으로 자주 쓰는 통계 3종이 최솟값·최댓값·개수입니다. 그리고 평균의 약점을 보완하는 또 하나의 대푯값이 중앙값입니다.

5-1. 최솟값 · 최댓값 · 개수

메서드	무엇을	왜 보나
<code>.min()</code>	가장 작은 값	데이터의 한쪽 끝 — 이상값 가능성 점검
<code>.max()</code>	가장 큰 값	과열·과부하 등 위험 신호 탐지
<code>.count()</code>	결측 아닌 값의 수	5번 측정과 500번 측정은 신뢰도가 다름

양 끝값 훑기

세 통계를 한 번에 확인

```
print(df.groupby("냉각기상태")["온도"].agg(["min", "max", "count"]))
```

```
▶           min    max  count
▶ 냉각기상태
▶ 고장   35.6   57.8     40
▶ 저하   44.0   51.2     40
▶ 정상   35.3   36.4     40
```

이 표는 표준편차만으로는 안 보이던 것을 보여줍니다. **고장 라인의 온도 범위가 35.6도에서 57.8도까지, 무려 22.2도**입니다. 평균이 54.67도인데 최솟값이 35.6도라니 이상합니다. 평균 근처에 대부분 몰려 있고 **몇 개가 아주 아래로 떨어져 있다**는 뜻입니다.

반면 정상 라인은 35.3도에서 36.4도, **범위가 1.1도밖에 안 됩니다**. 표준편차 0.36이 말해 주던 것을 양 끝값이 다시 확인해 줍니다.

설비 적용

양 끝값은 **이상값일 가능성이 가장 높은 자리**입니다. 데이터를 받으면 가장 먼저 훑어야 하는 이유가 여기 있습니다. 다만 이상값이 곧 오류는 아닙니다. **측정 오류인지, 진짜 설비 이상 신호인지 구분하는 것이 품질 확인의 첫 단계**입니다. 오늘 데이터의 고장 라인 최솟값 35.6도는 오류로 보기보다는 **사이클 초반의 낮은 온도가 섞였을 가능성**을 먼저 의심해야 합니다.

5-2. 중앙값 — 극단값에 흔들리지 않는 대푯값

메서드 `.median()` 크기순 한가운데 값

정의 값을 크기 순으로 줄 세웠을 때 정확히 한가운데 있는 값입니다. 개수가 짝수면 가운데 두 값의 평균을 씁니다.

형태 `df.groupby("기준열")["대상열"].median()`

```
# 중앙값 찾기
print(df.groupby("냉각기상태")["온도"].median().round(3))
```

▶ 냉각기상태

▶ 고장 55.45

▶ 저하 44.90

▶ 정상 35.90

▶ Name: 온도, dtype: float64

왜 쓰나 평균은 **극단값에 끌려갑니다**. 직원 아홉 명과 사장 한 명의 월급 평균을 내면 사장 연봉이 평균을 끌어올립니다. 그런데 중앙값은 흔들리지 않습니다. 순서만 보기 때문입니다.

응용 평균과 중앙값을 나란히 놓고 **그 차이**를 보는 것이 실전 사용법입니다. 두 값이 벌어지면 치우침이나 극단값이 있다는 신호입니다. 실습 5가 요구한 것이 정확히 이 비교입니다.

```
m = df.groupby("냉각기상태")["온도"].mean()
md = df.groupby("냉각기상태")["온도"].median()
print(pd.DataFrame({"평균": m.round(2), "중앙값": md.round(2), "차이": (m - md).round(2)}))
```

▶ 평균 중앙값 차이

▶ 냉각기상태

▶ 고장 54.67 55.45 -0.78

▶ 저하 45.46 44.90 0.56

▶ 정상 35.89 35.90 -0.01

개념

차이의 부호가 방향을 알려줍니다. 고장 라인은 평균이 중앙값보다 0.78도 **낮습니다**. 즉 아래로 튀는 낮은 값들이 평균을 끌어내리고 있습니다. 5-1에서 본 최솟값 35.6도가 바로 그 범인입니다. 반대로 저하 라인은 평균이 중앙값보다 0.56도 **높으니** 위로 튀는 값이 있다는 뜻입니다.

주의

정상 라인의 차이는 **0.01도**입니다. 사실상 0입니다. 평균과 중앙값이 이렇게 붙어 있다는 것은 **데이터가 한쪽으로 치우치지 않고 고르게 분포**한다는 뜻입니다. 실습 5의 코드 주석에 적힌 "정상: 35.89 vs 35.90"이 바로 이 관찰입니다.

5-3. 대푯값 두 개를 함께 쓰는 법

상황	평균과 중앙값	읽는 법	대응
거의 같음	차이 0.1 미만	고르게 분포함	평균을 그대로 믿어도 됨
평균 > 중앙값	평균이 위로	위로 튀는 값 존재	최댓값 확인
평균 < 중앙값	평균이 아래로	아래로 튀는 값 존재	최솟값 확인
크게 벌어짐	차이가 표준편차급	심한 치우침	중앙값을 대푯값으로

개념

두 값 비교만으로도 치우침을 진단할 수 있습니다. 히스토그램을 그리지 않고도, 숫자 두 개만 보면 분포의 모양을 짐작할 수 있다는 뜻입니다. 아직 시각화를 배우지 않은 지금 단계에서 이것은 상당히 강력한 도구입니다.

STEP 6

agg — 여러 통계를 한 번에

리스트 방식과 이름 붙이기 방식

여기까지 우리는 평균·분산·표준편차·중앙값·최대값을 배웠습니다. 그런데 이 다섯을 한 표에 담으려면 지금까지의 방법으로는 **다섯 줄을 따로 쓰고 눈으로 짝을 맞춰야** 합니다. 강사님도 같은 불편을 겪으며 코드를 치셨습니다.

불편한 방법 — 출력이 따로따로

```
# 앞선 내용처럼 냉각기상태별 평균 온도를 출력하기
print(df.groupby("냉각기상태")["온도"].mean().round(2))
```

```
# 냉각기상태별 온도의 표준편차를 출력하기
print(df.groupby("냉각기상태")["온도"].std().round(2))
```

```
▶ 냉각기상태
▶ 고장  54.67
▶ 저하  45.46
▶ 정상  35.89
▶
▶ 냉각기상태
▶ 고장  3.76
▶ 저하  1.47
▶ 정상  0.36
```

출력이 두 덩어리로 나뉘어 있어서 "고장의 평균은 54.67, 표준편차는 3.76"이라고 읽으려면 **눈이 위아래를 오가야** 합니다. 통계가 다섯 개면 다섯 덩어리가 됩니다. 이 문제를 해결하는 것이 **agg**입니다.

6-1. agg 리스트 방식

메서드 `.agg()` 여러 통계를 한 번에 구해 표로

정의 `aggregate`(집계하다)의 줄임말입니다. **통계 이름을 문자열 목록으로 묶어 넘기면** 각각을 계산해 하나의 표로 돌려줍니다.

형태 `df.groupby("기준열")["대상열"].agg(["통계1", "통계2", ...])`

```
# 냉각기상태별 온도의 평균과 표준편차를 한번에 정리하기
print(df.groupby("냉각기상태")["온도"].agg(["mean", "std"]).round(2))
```

	mean	std
▶ 냉각기상태		
▶ 고장	54.67	3.76
▶ 저하	45.46	1.47
▶ 정상	35.89	0.36

왜 쓰나 행이 그룹, 열이 통계인 표가 한 번에 나옵니다. 두 번 실행하고 눈으로 맞추던 것이 한 줄로 끝납니다. 게다가 이 결과는 데이터프레임이므로 뒤에 `.sort_values()`를 이어 붙일 수 있습니다.

응용 통계 이름을 늘리면 열이 늘어납니다. `mean · std · var · max · min · median · count`를 자유롭게 조합할 수 있습니다.

```
# 교대별 진동의 평균·표준편차·최댓값을 리스트로 한 번에
print(df.groupby("운전부하")["진동"].agg(["mean", "std", "max"]).round(3))
```

	mean	std	max
▶ 운전부하			
▶ 고부하	0.602	0.083	0.779
▶ 저부하	0.629	0.031	0.741

개념

통계 이름은 **따옴표로 감싼 문자열**입니다. `agg(["mean", "std"])`이지 `agg([mean, std])`가 아닙니다. 따옴표를 빼면 정의되지 않은 이름이라며 `NameError`가 납니다.

주의

위 응용 결과를 읽어 보십시오. 고부하와 저부하의 **평균 진동은 0.602와 0.629로 거의 같습니다**. 그런데 **표준편차는 0.083과 0.031로 2.7배 차이가 납니다**. 평균만 봤다면 "둘이 비슷하다"에서 끝났을 텐데, 표준편차를 함께 보니 **고부하 쪽이 훨씬 들쭉날쭉하다**는 사실이 드러납니다. 4장에서 배운 4분면으로 말하면 고부하는 2사분면, 불안정 주의입니다.

6-2. 이름 붙이기 방식 — named aggregation

리스트 방식에는 두 가지 아쉬움이 있습니다. 첫째, 열 이름이 `mean`, `std`처럼 영어로 나옵니다. 강사님 표현대로 **"일반인은 잘 모르겠다"**는 것이지요. 둘째, **모든 통계가 같은 열에만 적용**됩니다. 온도의 평균과 진동의 평균을 한 표에 담을 수 없습니다.

패턴 `agg(결과이름=( "원본열", "통계" ))` 통계마다 대상 열과 이름을 따로 지정

정의 `결과이름 = (원본열, 통계)` 형태로 넘기는 방식입니다. 등호 왼쪽이 결과 열 이름, 오른쪽 괄호 안이 (어느 열을, 어떤 통계로)입니다.

형태 `df.groupby("기준열").agg(이름=( "열", "통계" ), ...)`

```
# 위 방식으로 결과를 보니 mean, std... 일반인은 잘 모르겠다
# 그래서 다른 이름들로 항목이 나타나게 만들자
print(df.groupby("냉각기상태").agg(
    평균온도 = ("온도", "mean"),
    온도편차 = ("온도", "std"),
    평균진동 = ("진동", "mean"),
    측정수 = ("온도", "count"),
).round(2))
```

```
▶ 평균온도 온도편차 평균진동 측정수
▶ 냉각기상태
▶ 고장 54.67 3.76 0.69 40
▶ 저하 45.46 1.47 0.61 40
▶ 정상 35.89 0.36 0.55 40
```

왜 쓰나 열 이름이 **우리말로 자유롭게** 붙습니다. 그리고 온도와 진동을 **한 표에** 담았습니다. 리스트 방식으로는 불가능한 일입니다. 이 표 하나가 곧 설비 진단표가 됩니다.

응용 `groupby` 뒤에 **대상 열을 지정하지 않는다**는 점이 리스트 방식과 다릅니다. 어느 열을 쓸지는 각 괄호 안에서 정하기 때문입니다. 아래처럼 대상 열을 붙이면 오히려 오류가 납니다.

```
# 잘못된 예 — groupby 뒤에 열을 붙이면 안 된다
df.groupby("냉각기상태")["온도"].agg(평균=("온도", "mean"))

# 올바른 예 — 열을 붙이지 않는다
df.groupby("냉각기상태").agg(평균=("온도", "mean"))

▶ SpecificationError: nested renamer is not supported
▶
▶ 평균
▶ 냉각기상태
▶ 고장 54.668
▶ 저하 45.465
▶ 정상 35.885
```

개념

측정수 = ("온도", "count")에서 count는 온도 값이 몇 개인지 셉니다. 1장에서 배운 것처럼 결측은 빠집니다. 진단표에 측정수를 넣는 이유는 "이 평균이 몇 개를 근거로 나왔는지"를 표 안에서 바로 확인하기 위해서입니다.

주의

등호 왼쪽 이름에는 띄어쓰기를 쓸 수 없습니다. 평균 온도 = (...)는 문법 오류입니다. 평균온도처럼 붙여 쓰거나 밑줄을 쓰십시오.

6-3. 왜 이름 붙이기를 권하는가

방식	문법	장점	단점
리스트	<code>agg(["mean", "std"])</code>	짧고 간단	열 이름이 영어 · 한 열만
이름 붙이기	<code>agg(이름=("열", "통계"))</code>	이름 자유 · 여러 열 가능	조금 길다
딕셔너리 (옛날)	<code>agg({"온도": ["mean"]})</code>	—	일부 기능 제거됨

주의

인터넷의 오래된 예제 코드에서 중괄호 방식(`agg({"온도": ["mean", "std"]})`)을 자주 보게 됩니다. 판다스 초기에 쓰던 문법인데, 버전이 올라가면서 일부 기능이 제거되었습니다. 지금도 단순한 형태는 동작하지만, 열 이름을 새로 붙이는 기능은 사라졌습니다. 교안의 결론은 명확합니다. 처음부터 이름 붙이기 방식으로 익혀 두는 것이 안전합니다.

정리하면 이렇습니다. 통계를 여러 개 보고 싶는데 열이 하나라면 리스트 방식, 열이 여럿이거나 결과 이름을 우리말로 붙이고 싶다면 이름 붙이기 방식입니다. 실무 리포트를 만들 때는 후자를 쓰게 됩니다.

STEP 7

진단표 만들기과 그 함정

실습 1~5, 그리고 편차가 안 줄었다는 발견

교안 14_02의 실습은 다섯 문제입니다. 앞의 세 문제는 앞 장에서 배운 것을 그대로 쓰는 연습이고, **실습 4와 5에서 진단표라는 결과물**이 나옵니다. 그리고 그 진단표를 실제로 만들어 보면 교안이 예상하지 못한 결론이 나옵니다.

7-1. 실습 1 — 평균 · 분산 · 표준편차

Practice/14_02_example.py — 실습 1

```
df_1 = pd.read_csv("Data/14_hydraulic.csv", encoding="utf-8")
```

```
# 진동 열 전체의 평균·분산·표준편차
```

```
print("mean():", df_1["진동"].mean().round(3),  
      "|var():", df_1["진동"].var().round(3),  
      "|std():", df_1["진동"].std().round(3))
```

```
# 표준편차를 제공하면 분산과 같아지는지 확인
```

```
print("std() ** 2:", (df_1["진동"].std() ** 2).round(6),  
      "|var():", df_1["진동"].var().round(6))
```

```
▶ mean(): 0.616 |var(): 0.004 |std(): 0.064  
▶ std() ** 2: 0.00409 |var(): 0.00409
```

주의

첫 줄에서 **분산이 0.004로 보입니다**. round(3) 때문에 뭉개진 것입니다. 실제 값은 0.00409입니다. 둘째 줄에서 round(6)으로 자릿수를 늘리자 비로소 제곱 관계가 확인됩니다. **작은 값을 다룰 때는 반올림 자릿수가 곧 정보 손실입니다**. 이 코드를 쓴 사람이 자릿수를 다르게 잡은 것은 정확한 판단이었습니다.

실습 1 — 두 방식 비교

```
# agg에 리스트를 넘겨 두 통계를 한 번에
print(df_1.groupby("냉각기상태")["진동"].agg(["mean", "std"]).round(3))

# 판다스가 더 권장하는 방식으로 통계 항목별 이름 붙여 정리하기
print(df_1.groupby("냉각기상태")
      .agg(평균=("진동", "mean"), 표준편차=("진동", "std"))
      .round(3))
```

	mean	std
▶ 냉각기상태		
▶ 고장	0.688	0.048
▶ 저하	0.610	0.010
▶ 정상	0.549	0.009
▶		
▶ 평균 표준편차		
▶ 냉각기상태		
▶ 고장	0.688	0.048
▶ 저하	0.610	0.010
▶ 정상	0.549	0.009

숫자는 같고 **열 이름만 우리말로 바뀌었습니다**. 두 방식의 차이를 이보다 명확하게 보여주는 예는 없습니다. 그리고 여기서 진동의 흠어짐이 온도와 완전히 다른 양상을 보입니다. **고장 라인의 진동 편차 0.048은 저하·정상(0.010, 0.009)의 다섯 배입니다**. 온도에서 본 패턴과 같은 방향입니다.

7-2. 실습 2 — 합격과 불합격의 결정적 차이

실습 2는 품질검사 데이터를 씁니다. 합격품과 불합격품의 지표 통계를 비교하는 문제입니다.

Practice/14_02_example.py — 실습 2

```
df_2 = pd.read_csv("Data/14_hydraulic_qc.csv", encoding="utf-8")
feats = ["지표01", "지표05"]
```

```
# 판정 열로 그룹을 나눠 주요 지표들의 평균 집계
```

```
print(df_2.groupby("검사결과")[feats].mean().round(2))
```

```
# 같은 그룹 기준으로 표준편차를 구해 흩어짐 비교
```

```
print(df_2.groupby("검사결과")[feats].std().round(2))
```

```
▶ 지표01  지표05
▶ 검사결과
▶ 불합격  55.84  4.02
▶ 합격   35.91  6.69
▶
▶ 지표01  지표05
▶ 검사결과
▶ 불합격  1.32  1.37
▶ 합격   0.97  0.01
```

제출 코드의 주석이 이 결과를 정확하게 읽었습니다. "지표01과 지표05 모두 합격이 불합격보다 표준편차가 훨씬 작다. 합격품은 값이 일정한 범위에 몰려 있고, 불량품은 흩어져 있다는 뜻이다."

설비 적용

특히 **지표05의 표준편차가 합격 0.01 대 불합격 1.37로 137배** 차이 납니다. 합격품의 지표05는 사실상 한 값에 고정되어 있고, 불량품은 사방으로 흩어져 있다는 뜻입니다. 현장 언어로 옮기면 "공정이 제어되고 있으면 값이 고정되고, 제어를 벗어나면 값이 흩어진다"입니다. 이것이 **관리도(control chart)**의 기본 원리이고, 나중에 배울 이상 탐지 모델의 출발점이기도 합니다.

평균도 함께 보면 더 선명합니다. 지표01은 불합격 55.84 대 합격 35.91로 **평균 자체가 20이나 벌어졌습니다**. 반면 지표 05는 4.02 대 6.69로 평균 차이는 작지만 **편차 차이가 압도적입니다**. **평균으로 갈리는 지표와 편차로 갈리는 지표가 따로 있다는 뜻입니다**.

7-3. 실습 3 · 4 — 진단표 만들고 정렬하기

실습 4는 오늘 배운 것을 모두 모아 **설비 진단표**를 만드는 문제입니다. 이름 붙이기 방식의 진가가 여기서 드러납니다.

Practice/14_02_example.py — 실습 4

```
report_4 = (
```

```

df_4.groupby("밸브상태")
    .agg(
        측정수 = ("온도", "count"),
        평균온도 = ("온도", "mean"),
        온도편차 = ("온도", "std"),
        평균진동 = ("진동", "mean"),
        평균압력 = ("압력", "mean"),
    )
    .round(2)
)
print(report_4)

# 온도편차를 기준으로 내림차순 정렬
print(report_4.sort_values("온도편차", ascending=False))

```

```

> 측정수 평균온도 온도편차 평균진동 평균압력
> 밸브상태
> 경미  20  44.86  8.11  0.62  161.56
> 심각  19  46.02  8.40  0.63  163.39
> 정상  61  45.11  7.79  0.61  159.99
> 자연  20  45.86  8.93  0.62  161.27
>
> 측정수 평균온도 온도편차 평균진동 평균압력
> 밸브상태
> 자연  20  45.86  8.93  0.62  161.27
> 심각  19  46.02  8.40  0.63  163.39
> 경미  20  44.86  8.11  0.62  161.56
> 정상  61  45.11  7.79  0.61  159.99

```

코드를 괄호로 감싸 여러 줄로 나눈 방식에 주목하십시오. **바깥 괄호 안에서는 줄을 마음대로 바꿀 수 있습니다.** 메서드 체이닝이 길어질 때 이렇게 쓰면 읽기가 훨씬 편합니다. 이 코드를 쓴 사람이 잘한 부분입니다.

결과를 보면 교안이 예상한 대로 **자연의 온도편차 8.93이 최대**입니다. 정렬해서 맨 위에 올렸으니 "자연 밸브가 가장 불안정한 후보"라는 결론이 나옵니다. 실습의 정답은 여기까지입니다. **그런데 이 결론에는 큰 문제가 있습니다.**

7-4. 결정적 발견 — 편차가 전혀 줄지 않았다

개념

그룹으로 나누는 이유는 **그룹 안이 더 균질해지기를 기대하기** 때문입니다. 좋은 기준으로 나누면 **그룹 내 편차가 전체 편차보다 확실히 작아집니다.** 반대로 그룹 내 편차가 전체와 비슷하다면, 그 기준은 데이터를 전혀 설명하지 못한다는 뜻입니다.

이 잣대로 두 기준을 비교해 보겠습니다. **전체 온도의 표준편차는 8.04도**입니다. 이것이 기준선입니다.

기준	그룹	그룹 내 온도 표준편차	전체(8.04) 대비	판정
냉각기상태	고장	3.76	47%	
	저하	1.47	18%	잘 설명함
	정상	0.36	4%	
밸브상태	지연	8.93	111%	
	심각	8.40	104%	전혀 설명 못 함
	경미	8.11	101%	
	정상	7.79	97%	

숫자가 말하는 바는 분명합니다. 냉각기상태로 나누면 그룹 내 편차가 전체의 4~47%로 크게 줄어듭니다. 온도의 변동을 냉각기상태가 상당 부분 설명한다는 뜻입니다. 반면 밸브상태로 나누면 그룹 내 편차가 전체의 97~111%로 조금도 줄지 않습니다. 나누나 마나라는 뜻입니다.

자주 하는 실수

그러므로 "지연 밸브의 온도편차 8.93이 최대이므로 지연이 가장 불안정하다"는 결론은 성립하지 않습니다. 네 그룹의 편차가 7.79에서 8.93까지 고작 1.14 차이인데, 이는 전체 편차 8.04의 14%에 불과한 잡음 수준입니다. 정렬은 언제나 1등을 만들어 냅니다. 정렬해서 맨 위에 온 것이 곧 의미 있는 차이는 아닙니다.

평균온도로 확인하면 더 명확합니다. 밸브상태별 평균온도는 44.86에서 46.02까지 1.16도 차이입니다. 전체 표준편차 8.04도의 7분의 1입니다. 반면 냉각기상태별 평균온도는 35.89에서 54.67까지 18.78도 차이로, 전체 표준편차의 2.3 배입니다. 같은 데이터를 어떤 기준으로 자르느냐에 따라 이렇게 달라집니다.

설비 적용

현장 언어로 옮기면 이렇습니다. 온도를 보고 냉각기 상태를 짐작하는 것은 가능합니다. 54도면 고장, 45도면 저하, 36도면 정상입니다. 그러나 온도를 보고 밸브 상태를 짐작하는 것은 불가능합니다. 밸브가 정상이든 심각이든 온도 분포가 거의 같기 때문입니다. 2장에서 본 "밸브상태별 최고 온도가 네 그룹 모두 57도대"라는 관찰이 여기서 설명됩니다. 밸브 진단이 필요하다면 온도가 아닌 다른 측정값을 찾아야 합니다.

7-5. 실습 5 — 전체 기준선부터 진단표까지

실습 5는 오늘의 마지막 종합 문제입니다. **전체 통계로 기준선을 잡고 → 그룹별로 치우침을 확인하고 → 진단표를 만들어 정렬**하는 세 단계입니다. 7-4에서 본 "전체 표준편차를 기준선으로 삼는다"는 사고가 사실 이 문제의 첫 단계에 이미 들어 있었습니다.

Practice/14_02_example.py — 실습 5

```
# 1. 온도 열의 전체 평균과 표준편차로 기준선 파악
print("mean():", df_5["온도"].mean().round(2), "|std():",
      df_5["온도"].std().round(2))

# 2. 라인별 평균과 중앙값을 함께 구해 치우침 확인
print(df_5.groupby("냉각기상태")["온도"].agg(["mean", "median"]).round(2))

# 3. 설비 진단표를 온도편차 순으로 정렬해 우선 점검 대상 선정
report_5 = (df_5.groupby("밸브상태")
            .agg(평균온도=("온도", "mean"), 온도편차=("온도", "std"), 평균진동=("진동",
            "mean")))
            .round(2))
print(report_5.sort_values("온도편차", ascending=False))
```

```
▶ mean(): 45.34 |std(): 8.04
▶
▶      mean  median
▶ 냉각기상태
▶ 고장    54.67    55.45
▶ 저하    45.46    44.90
▶ 정상    35.89    35.90
▶
▶      평균온도  온도편차  평균진동
▶ 밸브상태
▶ 지연    45.86    8.93    0.62
▶ 심각    46.02    8.40    0.63
▶ 경미    44.86    8.11    0.62
▶ 정상    45.11    7.79    0.61
```

1단계에서 나온 전체 표준편차 8.04가 3단계 진단표의 8.93·8.40·8.11·7.79와 거의 같습니다. 문제는 이 비교를 하라고 명시하지 않았을 뿐입니다. 실습 5를 제대로 푼다는 것은 세 출력을 따로 내는 것이 아니라 **1단계의 기준선으로 3 단계를 판정하는 것**입니다.

STEP 8

상관관계 — 함께 움직이는가

corr · 부호와 절댓값 · 상관 행렬 · 상관 ≠ 인과

여기서 질문이 한 번 더 바뀝니다. 지금까지는 **열 하나를 요약**했습니다. 평균이 얼마인지, 얼마나 흩어졌는지. 이제부터는 **열 두 개 사이의 관계**를 봅니다. 온도가 오르면 진동도 함께 오르는가.

8-1. 함께 움직인다는 것

유압 시스템은 하나의 회로입니다. 온도·진동·압력이 같은 회로에서 나온 값이므로 서로 **연동될 가능성**이 있습니다. 온도가 오르면 냉각효율은 내려가는 식으로 **부호가 반대인** 쌍도 있습니다. 이 연동을 수치로 재는 것이 상관계수입니다.

방향	무슨 뜻	상관계수 부호	설비 예시
같은 방향	한쪽이 오르면 다른 쪽도 오름	양수 (+)	온도 ↑ → 진동 ↑
반대 방향	한쪽이 오르면 다른 쪽은 내림	음수 (-)	온도 ↑ → 냉각효율 ↓
관계 없음	한쪽이 변해도 다른 쪽은 무반응	0 근처	압력 ↔ 냉각효율

개념

상관계수는 **부호와 크기를 분리해서** 읽습니다. **부호는 방향, 절댓값은 강도**입니다. 이 두 문장이 상관계수 해석의 전부입니다. -0.9와 +0.9는 방향만 반대이고 **연결의 강도는 똑같습니다**. 반면 +0.1은 부호가 양수여도 **사실상 아무 관계가 없습니다**.

8-2. corr — 한 줄로 계산하는 상관계수

메서드 `.corr()` 두 열이 함께 움직이는 정도를 -1 ~ 1로

정의 `correlation`(상관관계)의 줄임말입니다. Series에 붙여 **다른 Series를 인자로 넘기면** 두 값의 상관계수를 돌려줍니다. 결과는 항상 **-1에서 1** 사이입니다.

형태 `df["열1"].corr(df["열2"])`

```
import pandas as pd

df = pd.read_csv("Data/14_hydraulic.csv", encoding="utf-8")

# 두 측정값의 상관계수
cor_1 = df["온도"].corr(df["진동"])
print(round(cor_1, 3))

print(round(df["온도"].corr(df["압력"]), 3))
print(round(df["진동"].corr(df["압력"]), 3))
```

-
- ▶ 0.931 ← 온도-진동, 매우 강한 양
 - ▶ 0.284 ← 온도-압력, 약한 양
 - ▶ 0.524 ← 진동-압력, 중간 양

왜 쓰나 "온도가 높은 사이클에서는 진동도 크다"라는 질문에 **숫자 하나로** 답합니다. 조건 필터링으로 나눠 비교하거나 그래프를 그리지 않아도 됩니다.

응용 부호가 음수인 쌍도 찾아보면 관계가 더 잘 보입니다. 냉각효율은 온도와 **정반대로** 움직입니다. 물리적으로 당연한 결과입니다. 냉각이 잘 되면 온도가 낮고, 냉각이 안 되면 온도가 높습니다.

```
print(round(df["온도"].corr(df["냉각효율"]), 3))
print(round(df["진동"].corr(df["냉각효율"]), 3))
print(round(df["압력"].corr(df["냉각효율"]), 3))
```

-
- ▶ -0.951 ← 온도가 오르면 냉각효율은 내려간다
 - ▶ -0.862 ← 진동도 같은 방향의 반대 관계
 - ▶ -0.131 ← 압력과는 거의 무관

개념

강사님은 변수 이름을 cor_1, cor_2처럼 번호를 붙여 여러 개 만드셨습니다. **여러 쌍을 동시에 비교하려면 결과를 각각 변수에 담아 두는 편이 편하기 때문**입니다.

주의

corr()는 **결측 행을 자동으로 제외**하고 계산합니다. 편리하지만 위험하기도 합니다. 결측이 많은 열끼리 비교하면 실제로는 몇 줄만 가지고 낸 상관계수일 수 있습니다. 상관계수를 볼 때는 항상 **몇 개로 계산했는지**를 함께 확인해야 합니다.

8-3. 상관 행렬 — 모든 조합을 한 번에

쌍마다 한 줄씩 쓰면 열이 네 개일 때 여섯 줄, 열이 열 개면 마흔다섯 줄이 됩니다. 강사님도 "개별적으로 이렇게 이렇게 비교하는 거 말고 좀 묶어서 3개를 같이 비교해 보자"며 다음 방법을 보여주셨습니다.

메서드 `.corr()` (데이터프레임) 모든 열 조합의 상관계수를 표로

정의 데이터프레임 전체에 붙이면 모든 수치형 열 조합의 상관계수를 정사각형 표(상관 행렬)로 돌려줍니다.

형태 `df[["열1", "열2", "열3"]].corr()`

```
# 임시로 자료를 만들어서 여러 개의 컬럼을 동시에 뽑아내기
num = df[["온도", "진동", "압력"]]
print(num.corr().round(3))
```

```
▶   온도  진동  압력
▶온도  1.000  0.931  0.284
▶진동  0.931  1.000  0.524
▶압력  0.284  0.524  1.000
```

왜 쓰나 한 번의 실행으로 모든 쌍을 봅니다. 어느 쌍이 강하게 연결되어 있는지 표에서 바로 눈에 띕니다.

응용 열을 늘리면 표가 커집니다. 냉각효율까지 넣으면 4×4 표가 됩니다. 음수가 섞이면 표가 훨씬 흥미로워집니다.

```
num4 = df[["온도", "진동", "압력", "냉각효율"]]
print(num4.corr().round(3))
```

```
▶   온도  진동  압력  냉각효율
▶온도  1.000  0.931  0.284 -0.951
▶진동  0.931  1.000  0.524 -0.862
▶압력  0.284  0.524  1.000 -0.131
▶냉각효율 -0.951 -0.862 -0.131  1.000
```

개념

대괄호가 두 겹인 점에 주의하십시오. `df[["온도", "진동", "압력"]]`은 세 열을 뽑아 데이터프레임을 만듭니다. 대괄호가 한 겹이면 Series라서 `.corr()`에 인자를 줘야 합니다. 어제 배운 대괄호 규칙이 그대로 적용됩니다.

8-4. 상관 행렬 읽는 네 가지 규칙

확인 위치	무엇이 보이냐	왜 그런가	어떻게 쓰냐
대각선	항상 1.000	자기 자신과의 상관	읽지 않고 건너뛰

확인 위치	무엇이 보이냐	왜 그런가	어떻게 쓰냐
대칭 칸	위와 아래가 같은 값	A-B와 B-A는 같음	한쪽 절반만 확인
절댓값 큰 칸	1이나 -1에 가까움	강한 연결	우선 검토 대상
0 근처 칸	0에 가까움	관계 약함	빠른 배제 후보

위의 3×3 표로 연습해 봅시다. 대각선은 모두 1.000이니 건너뛴니다. 대칭이니 **오른쪽 위 삼각형 세 칸만** 보면 됩니다. 그 세 칸은 0.931(온도-진동), 0.284(온도-압력), 0.524(진동-압력)입니다. **가장 큰 0.931이 우선 검토 대상**이고, **가장 작은 0.284는 배제 후보**입니다.

개념

대각선을 제외하고 **절댓값이 큰 칸에 집중하는 것**이 상관 행렬 읽기의 요령입니다. 4×4 표에서 실제로 읽을 칸은 여섯 개, 10×10 표에서는 마흔다섯 개입니다. 사람이 눈으로 마흔다섯 칸을 훑기는 어렵습니다. 그래서 다음 장의 실습 2에서 **반복문으로 자동 추출**하는 방법을 배웁니다.

8-5. 절댓값으로 판단하는 상관 강도

절댓값 구간	해석	오늘 데이터의 예
0.9 이상	매우 강한 상관	온도-진동 0.931 · 온도-냉각효율 -0.951
0.7 ~ 0.9	강한 상관	진동-냉각효율 -0.862
0.4 ~ 0.7	중간 상관	진동-압력 0.524
0.2 ~ 0.4	약한 상관	온도-압력 0.284
0.2 미만	거의 무상관	압력-냉각효율 -0.131

주의

교안 p8과 p16~p17의 표는 **구간과 예시가 서로 어긋나 있습니다**. 예컨대 온도-진동 0.931을 "약한 상관"으로, 진동-압력을 "0.052 음의 방향"으로 적어 두었는데, 실제 계산값은 **+0.524(중간 양의 상관)**입니다. 위 표는 제가 직접 계산한 값으로 다시 맞춘 것입니다. 대조표는 부록 B에 있습니다.

한 가지 덧붙이면, **이 구간표는 절대적인 기준이 아닙니다**. 분야와 측정 특성에 따라 같은 0.5도 다르게 해석됩니다. 물리 센서끼리는 0.9가 흔하지만 사람의 행동 데이터에서는 0.3만 나와도 강한 편입니다. 교안의 표현대로 **"강도 기준은 결론이 아니라 우선순위 도구"**입니다.

8-6. 상관관계는 인과관계가 아니다

오늘 수업에서 강사님이 가장 재미있게 설명하신 대목입니다. "내가 TV를 틀어서 중계를 보면 그 경기가 잘 나가다가도 안 된다"는 이야기였습니다. 내가 TV를 켜고 팀이 진 것 사이에는 아무 인과가 없습니다.

구분	상관관계가 말하는 것	상관관계가 말하지 않는 것
내용	두 측정값이 함께 움직인다는 관찰 결과	무엇이 원인인지에 대한 결론
성격	관찰	설명
역할	원인 탐색의 단서 · 질문 생성 도구	원인 확정

오늘 데이터의 온도-진동 0.931은 매우 강한 상관입니다. 그렇다고 "온도가 올라가서 진동이 커진다"고 단정할 수 없습니다. 가능한 설명이 최소한 셋입니다. ① 온도가 진동을 유발한다. ② 진동이 마찰을 만들어 온도를 올린다. ③ 부하 증가라는 제3의 원인이 온도와 진동을 동시에 올린다.

개념

원인을 단정하기 전에 확인할 세 가지입니다. ① 공통 원인 후보가 있는가 — 둘 다 끌어올리는 제3의 변수를 찾아봅니다. ② 공학적 작동 원리와 맞는가 — 유압 회로의 구조상 그 방향이 말이 되는지 대조합니다. ③ 실험이나 현장 지식으로 검증 가능한가 — 실제로 온도를 올려 보고 진동이 따라 오르는지 확인합니다. 상관 결과는 답이 아니라 좋은 질문입니다.

STEP 9

실습 1 · 2 — 강한 상관 쌍 찾기

상관 행렬 읽기와 이중 반복문

교안 14_03의 PART 01 실습은 두 문제입니다. 실습 1은 앞 장에서 배운 것을 그대로 쓰고, 실습 2에서 오랜만에 반복문이 등장합니다. 강사님도 "오랜만에 반복문 같은 거 쓰는 코드"라며 미리 예고하셨습니다.

9-1. 실습 1 — 상관계수와 상관 행렬 구하기

항목	내용
목표	두 측정값의 상관계수를 구하고 여러 열의 상관 행렬 읽기
단계 1	corr로 두 지표의 상관계수를 구해 부호와 절댓값 해석
단계 2	여러 지표 열을 골라 corr로 상관 행렬 생성
단계 3	대각선(항상 1)과 대칭 구조를 확인하고 절댓값 큰 칸 찾기
예상 결과	지표07-08 상관 -0.969, 네 열 상관 행렬 출력

Practice/14_03_example.py — 실습 1

```
import pandas as pd
import os

qc = pd.read_csv(os.path.join("Data", "14_hydraulic_qc.csv"), encoding="utf-8")

# 1. 두 지표의 상관계수를 구해 부호와 절댓값 해석
cor_1 = qc["지표07"].corr(qc["지표08"])
print("지표07-지표08 상관계수:", round(cor_1, 3))

# 2. 여러 지표 열을 골라 상관 행렬 생성
num = qc[["지표01", "지표05", "지표07", "지표08"]]
print(num.corr().round(3))
```

▶ 지표07-지표08 상관계수: -0.969

▶

	지표01	지표05	지표07	지표08
▶ 지표01	1.000	-0.895	-0.882	0.758
▶ 지표05	-0.895	1.000	0.999	-0.966
▶ 지표07	-0.882	0.999	1.000	-0.969
▶ 지표08	0.758	-0.966	-0.969	1.000

교안의 예상 결과 -0.969와 정확히 일치합니다. 부호가 음수이므로 방향은 반대, 절댓값이 **0.969**이므로 강도는 매우 강합니다. 즉 **지표07이 커지면 지표08은 거의 확실하게 작아집니다.**

행렬에서 더 놀라운 값은 **지표05-지표07의 0.999**입니다. 거의 완벽한 정비례입니다. 이 정도면 두 지표가 **사실상 같은 것을 다르게 재고 있을 가능성**이 큼니다. 강사님이 강조하신 대로 대각선(모두 1.000)은 건너뛰고 대칭이므로 한쪽 삼각형만 보면 됩니다.

설비 적용

상관계수가 0.99를 넘는 두 지표를 발견하면 실무에서는 **둘 중 하나를 버리는 것을 검토**합니다. 같은 정보를 두 번 넣는 셈이라 모델을 복잡하게만 만들기 때문입니다. 이것을 **다중공선성(multicollinearity)** 문제라고 하며, 나중에 회귀 모델을 배울 때 다시 만나게 됩니다. **상관 행렬을 먼저 보는 이유 중 하나가 바로 이 중복 지표 제거**입니다.

9-2. 실습 2 — 이중 반복문으로 강한 쌍 자동 추출

열이 열 개면 쌍이 45개입니다. 눈으로 훑기 어려우니 **코드가 대신 찾게** 합니다. 강사님은 이 코드를 두고 **"파이썬 여러 번 잡은 사람이나 좀 짜 보는 것"**이라고 하시면서도 끝까지 함께 만드셨습니다.

실습 2 — 이중 반복문으로 자동 추출

1. 여러 지표 열로 상관 행렬 만들기

```
cols = [f"지표{i:02d}" for i in range(1, 11)]  
corr_m = qc[cols].corr()
```

2. 이중 반복으로 대각선을 제외한 각 쌍의 상관계수 확인

```
pairs = []  
for i in range(len(cols)):  
    for j in range(i + 1, len(cols)):      # j가 i+1부터 → 대각선·중복 자동 제외  
        r = corr_m.iloc[i, j]  
        if abs(r) >= 0.4:                  # 3. 절댓값이 기준 이상인 쌍만 모으기  
            pairs.append((cols[i], cols[j], round(r, 3)))
```

큰 순서로 정렬 — 부호와 무관하게 강도로 줄 세우기

```
pairs.sort(key=lambda x: abs(x[2]), reverse=True)
```

```
print("절댓값 0.4 이상 쌍:", len(pairs), "개")  
for a, b, r in pairs[:5]:  
    print(f" {a} - {b} : {r}")
```

▶ 절댓값 0.4 이상 쌍: 42 개

▶ 지표01 - 지표10 : 0.999

▶ 지표05 - 지표07 : 0.999

▶ 지표03 - 지표08 : 0.993

▶ 지표01 - 지표04 : -0.983

▶ 지표05 - 지표09 : 0.982

코드를 네 조각으로 뜯어보기

조각	코드	하는 일
① 열 이름 생성	f"지표{i:02d}"	{i:02d}는 두 자리로 만들어 앞을 0으로 채우라는 뜻. 7 → "07"
② 범위 지정	range(1, 11)	1부터 10까지. 끝값 11은 포함되지 않음
③ 중복 제거	range(i + 1, len(cols))	j가 항상 i보다 큼 → 대각선과 반대쪽 절반이 자동 제외
④ 강도로 정렬	key=lambda x: abs(x[2])	세 번째 원소의 절댓값으로 정렬 → 음수도 공평하게 평가

개념

③번이 이 코드의 핵심입니다. 안쪽 반복문을 `range(i + 1, ...)`로 시작하면 `j`가 언제나 `i`보다 큼니다. 그래서 `i == j`인 대각선 칸(항상 1.000)과, 이미 본 쌍의 반대편(A-B를 봤으면 B-A)이 **자동으로 걸러집니다**. 8-4에서 배운 "대각선 제외, 한쪽 절반만" 규칙을 코드로 옮긴 것입니다. `if i != j`로 대각선만 막으면 45개가 아니라 90개가 나옵니다.

그리고 ④번의 `abs()`도 중요합니다. **절댓값 없이 정렬하면 -0.983처럼 강한 음의 상관**이 목록 맨 아래로 밀려납니다. 8-1에서 배운 "부호는 방향, 절댓값은 강도"를 그대로 코드로 옮긴 것입니다.

주의

교안 p20의 예상 결과는 **"절댓값 0.4 이상 쌍 3개"**이지만, 지표01~10 전체로 돌리면 실제로는 **42개**가 나옵니다. 45쌍 중 42쌍이 기준을 넘는 것입니다. 강사님도 수업 중에 **"자, 쪽 나가죠. 여러 개가 나옵니다"**라고 하셨습니다. 이 데이터의 열 대부분이 서로 강하게 연동되어 있어서 **0.4라는 기준선이 사실상 아무것도 걸러 내지 못합니다**. 실무에서는 기준을 0.8이나 0.9로 올려야 의미가 생깁니다.

기준선을 바꿔 가며 몇 개가 남는지 세어 보면 이 데이터의 성격이 한눈에 보입니다.

기준선을 올려 보기

```
for t in [0.4, 0.7, 0.9, 0.95, 0.99]:
    n = sum(1 for a, b, r in pairs if abs(r) >= t)
    print(f"기준 {t}: {n}쌍")
```

- ▶ 기준 0.4: 42쌍
- ▶ 기준 0.7: 39쌍
- ▶ 기준 0.9: 21쌍
- ▶ 기준 0.95: 14쌍
- ▶ 기준 0.99: 3쌍

기준을 0.99로 올려야 비로소 3쌍이 됩니다. 교안이 말한 "3개"는 아마 이 기준을 염두에 둔 것으로 보입니다. **어떤 기준선을 쓰느냐가 곧 분석의 결론을 정합니다**. 기준선을 밝히지 않은 상관 분석 결과는 신뢰하기 어렵습니다.

치트시트

오늘 나온 모든 도구를 한 표에

A-1. 행을 세는 도구

도구	문법	결과	결측
size()	df.groupby("A").size()	그룹별 행 수 (Series)	포함
count()	df.groupby("A")["B"].count()	결측 아닌 값의 수	제외
value_counts()	df["A"].value_counts()	값별 빈도, 내림차순	제외
len()	len(df[df["A"] == "고장"])	조건에 맞는 행 수 (정수)	포함

A-2. 통계량

메서드	무엇	단위	한 줄 해석
.mean()	평균	원래 단위	전체를 한 숫자로 요약
.var()	분산	단위의 제곱	흩어짐 — 해석은 어려움
.std()	표준편차	원래 단위	평균에서 평균적으로 이만큼 떨어져짐
.median()	중앙값	원래 단위	극단값에 안 흔들리는 대푯값
.min()	최솟값	원래 단위	아래쪽 끝 — 이상값 점검
.max()	최댓값	원래 단위	위쪽 끝 — 과열·과부하 탐지
.corr()	상관계수	없음 (-1~1)	두 열이 함께 움직이는 정도

A-3. agg 두 가지 방식

복사해 쓰는 골격

```
# 리스트 방식 — 열 하나에 통계 여러 개
df.groupby("냉각기상태")["온도"].agg(["mean", "std", "max"]).round(2)

# 이름 붙이기 방식 — 열마다 통계와 이름을 따로
df.groupby("냉각기상태").agg(
    평균온도 = ("온도", "mean"),
    온도편차 = ("온도", "std"),
    측정수 = ("온도", "count"),
).round(2)

# 정렬까지 이어 붙이기
report.sort_values("온도편차", ascending=False)
```

A-4. 상관 분석 골격

복사해 쓰는 골격

```
# 두 열
print(round(df["온도"].corr(df["진동"]), 3))

# 여러 열 — 상관 행렬
print(df[["온도", "진동", "압력"]].corr().round(3))

# 강한 쌍 자동 추출
cols = ["온도", "진동", "압력", "냉각효율"]
m = df[cols].corr()
for i in range(len(cols)):
    for j in range(i + 1, len(cols)):
        if abs(m.iloc[i, j]) >= 0.7:
            print(cols[i], cols[j], round(m.iloc[i, j], 3))
```

- ▶ 온도 진동 0.931
- ▶ 온도 냉각효율 -0.951
- ▶ 진동 냉각효율 -0.862

A-5. 오늘의 판단 기준 세 가지

질문	기준	판정
이 평균을 믿어도 되나	그룹 내 표준편차 가 전체 표준편차보다 확실히 작은가	작으면 믿어도 됨
이 그룹 나누기가 의미 있나	그룹별 평균 차이 가 전체 표준편차보다 큰가	크면 의미 있음
이 상관을 강하다고 할 수 있나	절댓값 이 0.7 이상인가	0.9 이상이면 매우 강함

개념

세 기준의 공통점은 **혼자서는 아무것도 판정할 수 없다**는 것입니다. 평균은 표준편차와 비교해야, 그룹 차이는 전체 편차와 비교해야, 상관계수는 강도 기준과 비교해야 의미가 생깁니다. **통계는 항상 무언가와 비교입니다.**

오류·합정 사전

제출 코드 교정 · 교안 예상 결과 대조

B-1. 제출 코드 교정

오늘 제출하신 두 파일은 **문법 오류 없이 정상 실행**되었습니다. 다만 논리상 손볼 곳이 두 군데 있습니다.

① 14_01_example.py 실습 1 — 수치형 열에 value_counts

문제 코드

```
# x 이렇게 쓰면
for col in list(df_1.columns):
    print(df_1[col].value_counts())
```

▶ ... 온도 62줄, 진동 80줄, 압력 101줄, 냉각효율 54줄이 쏟아짐

바로잡기

```
# ✓ 범주형 열만 골라서
for col in df_1.select_dtypes(include="object").columns:
    print(df_1[col].value_counts())
    print("-" * 30)
```

▶ 냉각기상태 고장 40 · 저하 40 · 정상 40
 ▶ 운전부하 고부하 60 · 저부하 60
 ▶ 밸브상태 정상 61 · 지연 20 · 경미 20 · 심각 19
 ▶ result 정상 67 · 고장 53

select_dtypes(include="object")는 **문자열 열만** 골라 줍니다. 어제 배운 대로 value_counts()는 **범주형 전용**입니다. 수치형은 pd.cut()으로 구간을 묶은 뒤에 세야 합니다. 실행이 되니 오류로 보이지 않지만, **출력 300줄 중 297줄이 쓸모없다면 그것도 버그**입니다.

② 14_01_example.py 실습 2 — round 자릿수

구분	코드	결과	문제
✗ 제출	.round(1)	합격 0.9 / 불합격 0.1	주석은 셋째 자리 라 적혀 있음
✓ 교정	.round(3)	합격 0.94 / 불합격 0.06	주석과 일치

자주 하는 실수

이 데이터는 **합격 188건 대 불합격 12건(94% 대 6%)의 심한 불균형 데이터**입니다. `round(1)`로 뭉개면 불합격이 **0.1**로 표시되어 마치 10%인 것처럼 보입니다. 실제로는 6%입니다. 어제 배운 대로 **불균형이 심할수록 자릿수를 넉넉히 잡아야 합니다**. 자릿수 선택이 곧 해석을 바꿉니다.

③ 14_02_example.py — 교정할 것 없음

실습 1~5 모두 정상 동작하며 주석의 관찰도 정확합니다. 특히 **실습 4의 "지연이 8.93으로 최대"**와 **실습 5의 "정상: 35.89 vs 35.90"**은 실제 출력과 정확히 일치합니다. 괄호로 감싸 메서드 체이닝을 여러 줄로 나눈 방식도 좋습니다.

주의

다만 실습 4의 결론 **"편차가 큰 설비를 불안정 후보로 확인 — 지연 8.93"**은 7-4에서 본 대로 **통계적으로 의미 없는 차이**입니다. 코드가 틀린 것이 아니라 **문제가 요구한 결론 자체에 함정이 있는** 경우입니다. 실습 답안으로는 이대로 맞지만, 실무 리포트라면 "전체 편차 8.04와 비교하면 유의미한 차이가 아님"이라는 한 줄을 반드시 덧붙여야 합니다.

B-2. 교안 예상 결과 대조

교안의 설명 문장과 실제 실행 결과가 어긋나는 곳을 모았습니다. **코드는 맞고 해설 문장의 라벨이 어긋난 경우가 대부분**입니다.

교안	교안이 말한 것	실제 실행 결과
14_02 p10	"저하 2.15 가장 안정 · 고장 0.13 가장 들쭉날쭉"	정상 0.126 이 가장 안정 · 고장 14.135 가 가장 들쭉날쭉 (라벨 뒤바뀜)
14_02 p11	"정상 3.76 ... 평균에서 평균적으로 약 10도 떨어져 있음"	3.76은 고장 의 표준편차이고, 약 3.8도
14_02 p14	" C 라인 · 주의 " (35.89 / 0.36)	그 값은 가장 안정적 . 불안정한 쪽은 54.67 / 3.76
14_02 p18	"저하 평균 45.46 ↔ 중앙값 44.9 — 약 2도 차이 "	실제 차이는 0.56도
14_03 p8	"온도-진동 0.931 약한 상관 "	0.931은 매우 강한 상관 (0.9 이상)
14_03 p8	"진동-압력 0.052 음의 방향 "	실제 +0.524 , 중간 양의 상관 (p10 코드와도 불일치)
14_03 p20	"절댓값 0.4 이상 쌍 3개 "	지표01~10 전체로 42개 . 3개가 되려면 기준 0.99

개념

교안 14_02 p10·p11·p14는 **같은 종류의 실수**를 반복합니다. 숫자는 맞는데 **어느 그룹의 숫자인지가 어긋난** 것입니다. 이런 자료를 볼 때 대응법은 하나뿐입니다. **직접 실행해 보는 것**. 오늘 배운 agg 한 줄이면 세 그룹의 평균·분산·표준편차를 동시에 확인할 수 있습니다.

B-3. 오늘 만나기 쉬운 오류 다섯 가지

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>agg([mean, std])</code>	NameError — 정의되지 않은 이름	<code>agg(["mean", "std"])</code>
<code>groupby("A") ["B"].agg(x=("B", "mean"))</code>	SpecificationError	<code>groupby("A").agg(x=("B", "mean"))</code>
<code>agg(평균 온도=("온도", "mean"))</code>	SyntaxError — 이름에 공백	<code>agg(평균온도=("온도", "mean"))</code>
<code>df["온도"].corr()</code>	인자가 없어 TypeError	<code>df["온도"].corr(df["진동"])</code>
<code>df["온도", "진동"].corr()</code>	KeyError — 대괄호 한 겹	<code>df[["온도", "진동"]].corr()</code>

주의

두 번째와 다섯 번째가 오늘 가장 흔합니다. **이름 붙이기 방식에서는 groupby 뒤에 열을 붙이지 않습니다**. 그리고 상관 행렬을 만들 때는 대괄호가 두 겹입니다. **어제 배운 "대괄호 하나는 Series, 두 개는 데이터프레임" 규칙을 다시 떠올리십시오**.

실습 하이라이트와 다음 시간

핵심 결론 정리 · 14_03 PART 02 예고

C-1. 오늘 얻은 다섯 가지 결론

#	결론	근거
1	평균만으로는 설비를 판단할 수 없습니다.	평균 78도가 같아도 76~80도와 50~105도는 전혀 다른 설비
2	냉각기상태는 온도를 잘 설명합니다.	그룹 내 편차 0.36~3.76 vs 전체 8.04 — 4~47%로 축소
3	밸브상태는 온도를 전혀 설명하지 못합니다.	그룹 내 편차 7.79~8.93 — 전체의 97~111%, 전혀 안 줄어듦
4	합격품은 값이 모이고 불량품은 흩어집니다.	지표05 표준편차 합격 0.01 vs 불합격 1.37 — 137배
5	온도와 진동은 강하게 연동됩니다.	상관계수 0.931 — 단, 원인은 아직 모름

설비 적용

2번과 3번을 나란히 놓으면 오늘의 실무적 교훈이 나옵니다. **같은 데이터, 같은 도구를 써도 어떤 열로 그룹을 나누느냐에 따라 결과가 완전히 달라집니다.** 정비 리포트에 "밸브상태별 온도 분석"을 넣는 것은 지면 낭비입니다. 반대로 냉각기상태별 온도는 그 자체로 진단 근거가 됩니다. **분석을 시작하기 전에 어떤 기준으로 나눌지 고르는 일이 분석의 절반입니다.**

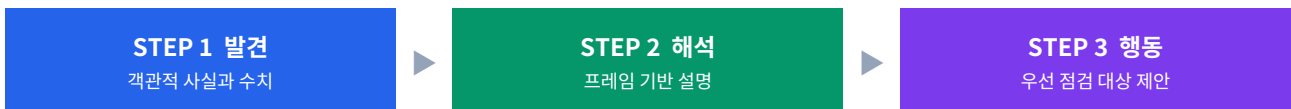
C-2. 오늘 푼 실습 정리

교안	실습	핵심 결과
14_01	4. groupby로 그룹 집계	밸브상태별 최고 온도가 네 그룹 모두 57도대로 동일
14_01	5. 그룹별 평균 비교와 정렬	진동 평균 차이가 0.02 — 정렬은 되지만 의미는 약함
14_01	6. 여러 기준 조합 그룹	저하-고부하 단 3건 · 정상-저부하 조합은 아예 없음
14_01	7. 빈도와 그룹 집계 종합	같은 53에 이르는 세 가지 길, 문제에 맞는 것은 len
14_02	1. 평균·분산·표준편차	$\text{std}^2 = \text{var}$ 확인 · round 자릿수가 곧 정보 손실

교안	실습	핵심 결과
14_02	2. 그룹별 통계 응용	합격품의 편차가 압도적으로 작음 — 관리도의 원리
14_02	3. agg로 여러 통계 한 번에	고부하와 저부하, 평균은 같고 편차는 2.7배 차이
14_02	4. agg 진단표 만들기	지연 8.93 최대 — 다만 전체 8.04와 비교하면 무의미
14_02	5. 그룹별 통계량 종합	전체 기준선으로 그룹 결과를 판정하는 흐름
14_03	1. 상관계수와 상관 행렬	지표07-08 -0.969 · 지표05-07은 0.999로 거의 동일
14_03	2. 강한 상관 쌍 찾기	기준 0.4에서 42쌍 — 기준선 선택이 결론을 정함

C-3. 다음 시간 예고 — 14_03 PART 02 집계 리포트

오늘 수업은 14_03의 상관관계 부분(실습 2)에서 끝났습니다. 남은 절반은 **집계 결과를 실제 정비 판단으로 연결하는** 법입니다.



남은 실습	내용	미리 계산해 본 결과
실습 3. 그룹별 상관 비교	합격·불합격 그룹에서 같은 지표 쌍의 상관이 달라지는지	전체 -0.969 · 합격 0.385 · 불합격 -0.998 (12건)
실습 4. 통합 리포트 종합	라인별 요약표 + 온도-진동 상관 + 라인별 고장 건수	상관 0.931 · 라인별 요약은 오늘 7장 진단표 그대로

주의

실습 3은 다음 시간 내용이지만 미리 계산해 보니 **교안 p25의 예상 결과(합격 0.482, 불합격 0.564)와 실제가 다릅니다.** 실제는 합격 **0.385**, 불합격 **-0.998**입니다. 부호까지 반대입니다. 더 중요한 것은 **불합격이 12건뿐**이라는 사실입니다. 12건으로 낸 -0.998은 우연히 나올 수 있는 값입니다. **실습 6에서 배운 "건수가 적은 그룹의 통계는 신중히"**가 여기서 다시 등장합니다.

그리고 전체가 -0.969인데 합격만 떼면 +0.385로 **부호가 뒤집힙니다.** 이것은 통계에서 유명한 함정인 **심슨의 역설**의 한 형태입니다. 전체를 볼 때와 그룹을 나눠 볼 때 결론이 정반대가 되는 현상입니다. 다음 시간에 이 지점을 눈여겨보시면 좋겠습니다.

C-4. 복습 순서

순서	무엇을	시간
1	Practice/14_02_example.py를 처음부터 다시 실행하며 출력 확인	15분
2	4장 4분면 표를 보고 세 라인이 어디에 속하는지 말로 설명해 보기	5분
3	7-4의 "전체 편차 대비 그룹 내 편차" 비교를 압력·진동 으로도 해 보기	15분
4	9-2의 이중 반복문을 14_hydraulic.csv의 네 수치열로 다시 짜 보기	15분
5	부록 B의 교안 대조표를 직접 실행해 확인하기	10분

C-5. 스스로 확인하는 질문 여섯 개

아래 여섯 질문에 자료를 보지 않고 답할 수 있으면 오늘 분량은 충분히 소화한 것입니다.

#	질문	확인 지점
1	size()와 count()는 무엇이 다른니까	결측을 세는가 빼는가 — 1-3
2	분산 대신 표준편차를 쓰는 이유는 무엇입니까	단위가 원래대로 돌아옴 — 4-5
3	평균과 중앙값이 벌어지면 무엇을 의심합니까	치우침 또는 극단값 — 5-2
4	agg 두 방식은 각각 언제 씁니까	열이 하나인가 여럿인가 — 6-3
5	상관계수 -0.95와 +0.3 중 어느 쪽이 강합니까	부호가 아니라 절댓값 — 8-1
6	그룹으로 나눈 것이 의미 있는지 어떻게 압니까	그룹 내 편차가 줄었는가 — 7-4

연결

6번이 오늘 처음 등장한 사고입니다. 나머지 다섯은 도구의 사용법이지만, **6번은 도구를 쓴 결과를 판정하는 기준**입니다. 이 질문에 답할 수 있으면 앞으로 어떤 데이터를 받아도 "어떤 열로 나눌 것인가"를 스스로 정할 수 있습니다.

오늘의 성취

평균 옆에 흠어짐을, 흠어짐 옆에 관계를

오늘 배운 도구는 일곱 개입니다. `size`, `count`, `var`, `std`, `median`, `agg`, `corr`. 그런데 도구보다 중요한 것이 **하나의 사고 습관**입니다.

개념

숫자 하나를 보면 반드시 비교 대상을 찾는다.

평균 54.67도를 보면 → 표준편차는 얼마인가를 묻습니다. 그룹 편차 8.93을 보면 → 전체 편차는 얼마인가를 묻습니다. 상관계수 0.931을 보면 → 이게 강한 편인가를 묻습니다.

오늘 수업에서 제일 중요한 발견인 "밸브상태는 온도를 설명하지 못한다"도 이 습관에서 나왔습니다. **진단표의 8.93을 전체의 8.04와 비교했기 때문에** 보인 것입니다.

한 걸음 더 나아가면, 이 사고 습관이 곧 **머신러닝의 출발점**입니다. "이 변수로 나누면 대상이 균질해지는가"라는 질문은 의사결정나무가 가지를 뻗을 때 던지는 질문과 정확히 같습니다. "두 변수가 중복되는가"라는 질문은 특성 선택의 첫 단계입니다. **오늘 손으로 계산한 것을 나중에는 모델이 대신 합니다.** 그래서 지금 손으로 해 보는 것이 의미가 있습니다.

어제까지	오늘 더한 것	다음에 만날 것
몇 건인가 세기	그 평균을 믿어도 되는지 판정하기	발견·해석·행동 리포트로 쓰기
<code>value_counts</code>	<code>var</code> · <code>std</code> · <code>median</code> · <code>agg</code>	<code>groupby</code> + <code>corr</code> 통합
그룹마다 평균 내기	두 값이 함께 움직이는지 보기	그룹마다 상관이 달라지는지 보기

마지막으로 오늘 확인한 숫자 하나만 다시 짚고 마치겠습니다. **정상 라인의 온도 표준편차는 0.36도입니다.** 120번 측정하는 동안 온도가 평균에서 0.36도밖에 벗어나지 않았다는 뜻입니다. 잘 관리되는 설비의 모습이 이렇습니다. 그리고 **고장 라인은 3.76도로 열 배입니다.** 이 열 배 차이를 읽어 내는 것이 오늘 배운 전부입니다.

— 21일차 정리 끝. 다음은 14_03 PART 02 집계 리포트입니다.